



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Norme di Progetto

Stato	Approvato
Versione	2.0.0
Autori	Davide Lorenzon Aldo Bettega Filippo Guerra Ana Maria Draghici
Verificatori	Ana Maria Draghici Davide Lorenzon Aldo Bettega Filippo Guerra Felician Mario Neculescu
Uso	Interno
Destinatari	Tutto il gruppo

Vers.	Data	Autore	Verificatore	Descrizione
2.0.0	2026-05-20	Filippo Guerra	Filippo Guerra	Approvazione
1.4.0	2026-05-18	Ana Maria Draghici	Filippo Guerra	Spostato da Pdq e integrato la sezione di automiglioramento Sezione 4.4
1.3.2	2026-05-21	Davide Lorenzon	Ana Maria Draghici	Aggiornamento dell'infrastruttura.
1.3.1	2026-05-19	Davide Lorenzon	Ana Maria Draghici	Aggiunta definition of done per le issue relative alla codifica.
1.3.0	2026-05-18	Aldo Bettega	Ana Maria Draghici	Aggiunte sezioni di codifica e progettazione
1.2.0	2026-05-03	Felician Mario Neculescu	Ana Maria Draghici	Aggiunti Specifica Tecnica e Manuale Utente alla sezione Sezione 3.2.6.3
1.1.0	2026-04-21	Filippo Guerra	Felician Mario Neculescu	Aggiunta alla sezione Sezione 2.3 la parte di Naming Convention
1.0.0	2026-04-01	Felician Mario Neculescu	Aldo Bettega	Approvazione
0.13.1	2026-03-20	Davide Lorenzon	Filippo Guerra	Aggiornamento della sezione relativa alla gestione dell'infra-struttura Sezione 4.3 • Numerazione automatica, • Tracciamento automatico, • Plot diagrammi, • Uso di script python,
0.13.0	2026-03-20	Ana Maria Draghici	Felician Mario Neculescu	Aggiunta alla Sezione 3.5 la de-

Vers.	Data	Autore	Verificatore	Descrizione
				scrizione delle varie strategie di testing (Processo di Qualifica)
0.12.0	2026-02-05	Davide Lorenzon	Filippo Guerra	Aggiunte descrizioni delle metriche di qualità
0.11.0	2026-01-14	Ana Maria Draghici	Davide Lorenzon	Completata prima scrittura Norme di Progetto
0.10.0	2026-01-10	Ana Maria Draghici	Davide Lorenzon	Rivisto processo di supporto <u>Sezione 3.1</u> , aggiunte sottosezioni <u>Sezione 3.4</u> e <u>Sezione 3.5</u>
0.9.0	2026-01-10	Ana Maria Draghici	Davide Lorenzon	Rivisti processi primari <u>Sezione 2.1</u> , sottosezioni <u>Sezione 2.2</u> e <u>Sezione 2.3</u>
0.8.4	2025-12-15	Davide Lorenzon	Ana Maria Draghici	Stesura della sezione del Processo di miglioramento <u>Sezione 4.4</u> e processo di formazione <u>Sezione 4.5</u>
0.8.3	2025-12-14	Davide Lorenzon	Ana Maria Draghici	Stesura della sezione Gestione dell'infrastruttura <u>Sezione 4.3</u>
0.8.2	2025-12-13	Davide Lorenzon	Ana Maria Draghici	Rivista introduzione, approfondita Gestione del processo <u>Sezione 4.2</u>
0.8.1	2025-12-10	Davide Lorenzon	Ana Maria Draghici	Aggiunta Rendicontazione delle ore <u>Sezione 4.2.4.2</u>

Vers.	Data	Autore	Verificatore	Descrizione
0.8.0	2025-12-09	Filippo Guerra	Davide Lorenzon	Aggiunto Processo di fornitura <u>Sezione 2.2</u>
0.7.0	2025-12-04	Aldo Bettega	Davide Lorenzon	Aggiunta scrittura commit <u>Sezione 3.4.4.1</u> , e sezione su workflow documentale <u>Sezione 3.2.4.3</u> , aggiornata definition of done <u>Sezione 3.5.3.5</u>
0.6.1	2025-12-02	Davide Lorenzon	Davide Lorenzon	Apportate modifiche di ordine nella documentazione <u>Sezione 3.2</u>
0.6.0	2025-12-02	Ana Maria Draghici	Aldo Bettega	Completata struttura documenti <u>Sezione 3.2.6.3</u> , aggiunto «Configurazione» <u>Sezione 3.3</u>
0.5.0	2025-11-30	Ana Maria Draghici	Aldo Bettega	Aggiunto «Definition of Done» <u>Sezione 3.5.3.5</u> e «Issue tracking System» <u>Sezione 4.2.4.4</u>
0.4.0	2025-11-29	Guerra Filippo	Ana Maria Draghici	Aggiunto «ruolo-documento» <u>Sezione 4.2.4.3</u>
0.3.0	2025-11-11	Ana Maria Draghici	Davide Lorenzon	Aggiunta struttura Analisi Requisiti <u>Sezione 3.2.6.4</u>
0.2.0	2025-11-11	Davide Lorenzon	Ana Maria Draghici	Aggiunta struttura dei documenti <u>Sezione 3.2.6.3</u>
0.1.0	2025-11-	Davide Lorenzon	Ana Maria Draghici	Stesura iniziale

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	1
1.4.1) Riferimenti normativi	1
1.4.2) Riferimenti informativi	2
2) Processi Primari	3
2.1) Introduzione ai processi primari	3
2.2) Processo di fornitura	3
2.2.1) Introduzione	3
2.2.2) Scopo del processo	3
2.2.3) Attività del processo	4
2.2.4) Procedure operative	4
2.2.4.1) Metodi di comunicazione	4
2.2.5) Documenti principali	5
2.2.6) Strumenti a supporto	5
2.3) Processo di sviluppo	6
2.3.1) Introduzione	6
2.3.2) Scopo del processo	6
2.3.3) Attività del processo	7
2.3.4) Inquadramento del processo nelle Baseline di progetto	7
2.3.5) Procedure operative	8
2.3.5.1) Documenti principali	10
2.3.6) Progettazione	10
2.3.6.1) Descrizione e scopo	10
2.3.6.2) Standard e linee guida da seguire	10
2.3.6.2.1) Diagrammi UML	10
2.3.6.2.2) Design Pattern	11
2.3.6.2.3) Principi di progettazione	11
2.3.6.3) Strumenti per la progettazione	11
2.3.6.4) Procedure di progettazione	12
2.3.6.4.1) Documentazione delle scelte progettuali	12
2.3.6.4.2) Aggiornamento della documentazione	12
2.3.7) Codifica	12
2.3.7.1) Descrizione e scopo	12
2.3.8) Strumenti a supporto	12
2.3.9) Convenzioni di stile	13
2.3.9.1) Buone pratiche di programmazione	13
2.3.9.1.1) Principi di qualità del codice	13

2.3.9.1.2)	Struttura del codice	14
2.3.9.1.3)	Gestione degli errori	14
2.3.9.2)	Formattazione e controllo qualità del codice	14
2.3.9.2.1)	Strumenti per il backend (Python)	15
2.3.9.2.2)	Strumenti per il frontend (js + Vue)	15
2.3.9.2.3)	Automazione tramite CI/CD	15
2.3.9.2.4)	Esecuzione locale	16
3)	Processi di Supporto	17
3.1)	Introduzione ai processi di supporto	17
3.2)	Processo di documentazione	17
3.2.1)	Introduzione	17
3.2.2)	Scopo del processo	17
3.2.3)	Attività del processo	18
3.2.3.1)	Impegno del gruppo	18
3.2.3.2)	Attività principali	18
3.2.4)	Procedure operative	18
3.2.4.1)	Identificazione dei documenti	18
3.2.4.2)	Progettazione dei documenti	19
3.2.4.3)	Workflow documentale	19
3.2.4.4)	Stati del documento	19
3.2.4.5)	Procedura di avanzamento tra stati	20
3.2.4.6)	Procedura di archiviazione e tracciabilità	20
3.2.4.7)	Pubblicazione e Distribuzione della Documentazione	21
3.2.5)	Inquadramento del processo nelle Baseline di progetto	21
3.2.6)	Documentazione fornita	21
3.2.6.1)	Elenco dei documenti	21
3.2.6.2)	Informazioni comuni	22
3.2.6.3)	Struttura specifica	22
3.2.6.4)	Analisi dei Requisiti	23
3.2.6.4.1)	Struttura principale	23
3.2.6.5)	Piano di Qualifica	23
3.2.6.5.1)	Struttura principale	23
3.2.6.6)	Piano di Progetto	24
3.2.6.6.1)	Struttura principale	24
3.2.6.7)	Verbali	25
3.2.6.7.1)	Struttura principale	25
3.2.6.8)	Diario di Bordo	26
3.2.6.9)	Norme di Progetto	26
3.2.6.9.1)	Struttura principale	26
3.2.6.10)	Glossario	27
3.2.6.11)	Specifica Tecnica	27

3.2.6.11.1) Struttura principale	27
3.2.6.12) Manuale Utente	28
3.2.6.12.1) Struttura principale	28
3.2.7) Strumenti di supporto	28
3.3) Processo di Gestione delle Configurazioni	29
3.3.1) Introduzione	29
3.3.2) Scopo del processo	29
3.3.3) Attività del processo	29
3.3.4) Procedure operative	29
3.3.4.1) Codice di versione	29
3.3.4.1.1) Regole di incremento	30
3.3.4.2) Versionamento automatico tramite Typst	30
3.3.4.3) Registro modifiche	31
3.3.4.4) Formato nome dei verbali	31
3.3.5) Strumenti principali utilizzati	31
3.4) Processo di Accertamento della Qualità	31
3.4.1) Introduzione	31
3.4.2) Scopo del processo	32
3.4.3) Attività del processo	32
3.4.4) Procedure operative	32
3.4.4.1) Scrittura dei commit	32
3.4.5) Strumenti	33
3.4.6) Metriche	33
3.5) Processo di Qualifica	33
3.5.1) Introduzione	33
3.5.2) Scopo del processo	33
3.5.3) Attività del processo	33
3.5.3.1) Analisi statica	34
3.5.3.1.1) Checklist di ispezione	34
3.5.3.1.2) Quando usare quale metodo	35
3.5.3.2) Analisi dinamica	35
3.5.3.3) Classificazione dei test	35
3.5.3.3.1) Stati dei test	36
3.5.3.3.2) Test di Unità	36
3.5.3.3.3) Test di Integrazione	37
3.5.3.3.4) Test di Sistema	37
3.5.3.3.5) Test di Accettazione	37
3.5.3.3.6) Test di Regressione	38
3.5.3.4) Validazione	38
3.5.3.4.1) Attività di validazione	38
3.5.3.5) Definition of Done	38

3.5.3.5.1)	Definition of Done - Ambito documentale	39
3.5.3.5.2)	Definition of Done - Ambito codice	39
3.5.4)	Strumenti a supporto	40
3.5.5)	Documentazione a supporto	40
4)	Processi Organizzativi	42
4.1)	Introduzione	42
4.2)	Processo di gestione del processo	42
4.2.1)	Introduzione	42
4.2.2)	Scopo del processo	42
4.2.3)	Attività del processo	43
4.2.4)	Procedure operative	44
4.2.4.1)	Ruoli di Progetto	44
4.2.4.2)	Rendicontazione delle ore	45
4.2.4.3)	Assegnazione Ruolo-Documento	45
4.2.4.4)	Issue Tracking System	46
4.2.4.4.1)	Creazione e struttura di un'issue	47
4.2.4.4.1.1)	Creazione e struttura di un'issue - Documentazione	47
4.2.4.4.1.2)	Creazione e struttura di un'issue - Codice	48
4.2.4.4.2)	Flusso operativo	50
4.3)	Processo di Gestione dell'infrastruttura	50
4.3.1)	Introduzione	50
4.3.2)	Scopo del processo	50
4.3.3)	Attività del processo	51
4.3.4)	Procedure operative	51
4.3.4.1)	Implementazione del processo	51
4.3.4.2)	Creazione	52
4.3.4.3)	Manutenzione	55
4.3.4.3.1)	Project board	55
4.3.4.3.2)	Milestone	55
4.3.4.4)	GitHub Actions	55
4.3.4.5)	Script e automazioni	55
4.3.4.6)	Discord	56
4.3.4.7)	Strumenti Google	56
4.4)	Processo di miglioramento	56
4.4.1)	Introduzione	56
4.4.2)	Attività del processo	57
4.4.3)	Procedure operative	57
4.4.3.1)	Ciclo PDCA	57
4.4.3.1.1)	Guida alla Retrospettiva	58

4.4.3.1.2)	Automiglioramento sulla Requirements and Technology Baseline	59
4.4.3.1.3)	Automiglioramento sulla Technology Baseline .	60
4.5)	Processo di formazione	60
4.5.1)	Introduzione	60
4.5.2)	Scopo del processo	60
4.5.3)	Attività del processo	60
4.5.4)	Procedure operative	61
4.5.4.1)	Apprendimento libero	61
4.5.4.2)	Apprendimento in gruppo	61
4.5.4.3)	Trasferimento dei ruoli	61
4.5.4.4)	Rendicontazione delle ore	61
5)	Metriche	63
5.1)	Introduzione	63
5.1.1)	Nomenclatura delle Metriche	65
6)	Metriche di Qualità del Processo	65
6.1)	Processi primari	65
6.1.1)	Fornitura	66
6.1.2)	Sviluppo	70
6.2)	Processi di supporto	71
6.2.1)	Documentazione	71
6.2.2)	Verifica	72
6.3)	Processi organizzativi	72
6.3.1)	Gestione dei processi	73
7)	Metriche di Qualità del Prodotto	74
7.1)	Funzionalità	74
7.2)	Affidabilità	76
7.3)	Usabilità	77
7.4)	Efficienza	78
7.5)	Manutenibilità	80

Elenco delle Tabelle

Tabella 1	Metodologie di testing	35
Tabella 2	Tipi di test	36
Tabella 3	Stati dei test	36
Tabella 4	Tipi di test d integrazione	37
Tabella 5	Strumenti utilizzati	52
Tabella 6	Automiglioramento: Requirements and Technology Baseline	59
Tabella 7	Automiglioramento: Technology Baseline	60
Tabella 8	Metriche processo di Fornitura	66
Tabella 9	Metriche processo di Sviluppo	70
Tabella 10	Metriche processo di Documentazione	71
Tabella 11	Metriche processo di Verifica	72
Tabella 12	Metriche processo di Gestione dei Processi	73
Tabella 13	Metriche di funzionalità del prodotto	74
Tabella 14	Metriche di affidabilità del prodotto	76
Tabella 15	Metriche di usabilità del prodotto	77
Tabella 16	Metriche di efficienza del prodotto ¹	78
Tabella 17	Metriche manutenibilità del prodotto	80

¹I valori soglia di questa sezione sono espressi in termini assoluti, non come percentuali relative.

1) Introduzione

1.1) Scopo del documento

In questo documento il gruppo definisce e tiene traccia del proprio way of working, ovvero l'insieme di pratiche e convenzioni adottate per organizzare e svolgere il lavoro di progetto.

La stesura del documento avviene in modo incrementale, evolvendosi parallelamente all'avanzamento delle attività. Nel corso del progetto potrà essere soggetto ad aggiunte, modifiche o rimozioni, derivate dal processo di apprendimento e sperimentazione del team.

Tali aggiornamenti nascono dall'analisi e dall'evoluzione delle best practices comuni nell'ingegneria del software, che il gruppo studia, applica e adatta in funzione delle esigenze del team e del contesto progettuale.

1.2) Scopo del prodotto

Il prodotto sviluppato è un'applicazione software progettata per verificare in modo automatico la conformità alla norma EN 18031, lo standard tecnico europeo che definisce i requisiti di sicurezza informatica per i dispositivi radio wireless (es. Wi-Fi, LTE, Bluetooth, IoT wireless).

L'obiettivo dell'applicazione è supportare e guidare l'utente nella valutazione dei requisiti di cybersecurity, eseguendo in autonomia i percorsi decisionali tramite decision tree (alberi di decisione). Questo approccio permette di accelerare, standardizzare e rendere più affidabile il processo di verifica della conformità, con la generazione automatica della documentazione tecnica richiesta a supporto dell'assessment.

1.3) Glossario

Per garantire precisione terminologica senza compromettere la leggibilità, in questo documento viene adottato il seguente approccio per la gestione dei riferimenti al Glossario:

I termini tecnici vengono marcati con **pedice "G" (esempio_G)**.

Questo sistema consente di mantenere il documento tecnicamente rigoroso, chiaro e facilmente navigabile, favorendo la consultazione mirata del Glossario solo quando necessario.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl

<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C1.pdf>

Ultima consultazione: 8 aprile 2026

- Regolamento del progetto
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultima consultazione: 8 aprile 2026

1.4.2) Riferimenti informativi

- Standard ISO/IEC/IEEE 12207:2017
<https://www.iso.org/standard/63712.html>
Ultima consultazione: 10 gennaio 2026
- Software Engineering, Ian Sommerville
https://galileodiscovery.unipd.it/permalink/39UPD_INST/1j3btfv/alma9938989417806046
Ultima consultazione: 20 febbraio 2026
- Standard ISO/IEC 9126
https://en.wikipedia.org/wiki/ISO/IEC_9126
Ultima consultazione: 10 gennaio 2026
- Standard ISO/IEC/IEEE 12207:1995
https://www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf
Ultima consultazione: 10 gennaio 2026
- Glossario, versione 2.0.0
<https://grouprubberduck.github.io/Documentazione/output/PB/DocumentazioneInterna/Glossario-v2.0.0.pdf>
Ultima consultazione: 20 marzo 2026

2) Processi Primari

2.1) Introduzione ai processi primari

Per sviluppare un sistema software di qualità, la semplice scrittura di codice e l'esecuzione dei test non sono sufficienti: il prodotto deve essere **affidabile, utilizzabile da un ampio numero di utenti e facilmente manutenibile nel tempo**.

Per raggiungere questi obiettivi, è necessario adottare un modello di sviluppo che definisca **processi chiari e strutturati**, capaci di guidare in modo sistematico le attività del progetto.

Secondo lo **standard ISO/IEC 12207**, i principali processi primari sono:

- **Fornitura**, che gestisce i rapporti con il committente o proponente esterno;
- **Sviluppo**, che si occupa della realizzazione del prodotto software vero e proprio.

2.2) Processo di fornitura

2.2.1) Introduzione

Il processo di fornitura descrive le attività necessarie per **instaurare, gestire e mantenere il rapporto tra il gruppo e la proponente esterna (BlueWind S.r.l.)** durante lo svolgimento del progetto.

Seguendo le linee guida dettate dai principi espressi dallo [Standard ISO 12207:2017](#) — che definisce il processo di supply come l'insieme delle attività volte alla definizione degli accordi contrattuali, alla pianificazione, al monitoraggio e alla consegna dei prodotti software — il gruppo si impegna a stabilire modalità **organizzate, verificabili e tracciabili** per comunicare, consegnare documentazione e garantire trasparenza verso la proponente.

2.2.2) Scopo del processo

Lo scopo del processo di fornitura è garantire che la relazione tra gruppo e proponente sia **strutturata, efficiente e basata su comunicazioni chiare e documentate**.

Gli obiettivi principali includono:

- Definire i **canali ufficiali di comunicazione** tra gruppo e proponente;
- Stabilire **frequenza, modalità e tempi dei confronti**;
- Assicurare che le richieste della proponente siano **comprese, tracciate e integrate** nel flusso di lavoro;
- Produrre **documentazione coerente** per revisioni e fasi di progetto;
- Garantire **trasparenza, tracciabilità e rispetto delle scadenze** previste da capitolato e milestone.

2.2.3) Attività del processo

Il processo di fornitura comprende le seguenti attività principali:

1. **Inizializzazione**

Analisi delle richieste della proponente, identificazione dei requisiti contrattuali e valutazione dei vincoli organizzativi.

2. **Preparazione delle risposte**

Realizzazione di eventuali contro-proposte basate sull'Analisi dei Requisiti (AdR).

3. **Contrattazione**

Confronto con la proponente per formalizzare requisiti, scadenze e modalità di lavoro.

4. **Pianificazione**

Definizione dell'organizzazione, del ciclo di vita del software, assegnazione delle risorse e gestione dei rischi.

5. **Esecuzione e controllo**

Implementazione delle attività pianificate, monitoraggio della qualità e del progresso del lavoro.

6. **Revisione e valutazione**

Raccolta di feedback dalla proponente per eventuali correzioni o aggiornamenti del lavoro.

7. **Consegna e completamento**

Rilascio dei prodotti software e della documentazione, garantendo supporto post-consegna.

2.2.4) Procedure operative

Per garantire continuità e tracciabilità nelle comunicazioni, il gruppo utilizza modalità **sincrone e asincrone**:

2.2.4.1) Metodi di comunicazione

Comunicazione sincrona (meeting):

- **Riunioni con la proponente (BlueWind S.r.l.):**

- Programmate periodicamente secondo le esigenze del progetto.
- Documentate tramite verbali esterni.
- Decisioni, richieste e dubbi vengono registrati come issue su GitHub.

- **Riunioni interne al gruppo:**

- Programmate per coordinare attività e aggiornare lo stato del progetto.
- Documentate tramite verbali interni.

- Decisioni operative e compiti interni tracciati come issue su GitHub.

Comunicazione asincrona:

- **Con la proponente:** email ufficiale, messaggistica (Telegram) per chiarimenti rapidi.
- **Internamente al gruppo:** chat interne (Discord/WhatsApp) per coordinamento e aggiornamenti.

Tracciamento

- Verbali esterni e interni;

Disponibilità

Il gruppo si impegna a fornire aggiornamenti regolari e materiali richiesti entro le scadenze di sprint e milestone. Il processo di fornitura produce documentazione fondamentale per la tracciabilità e la trasparenza del progetto.

2.2.5) Documenti principali

- **Verbali esterni:** resoconto dei meeting con BlueWind;
- **Verbali interni:** documentazione di riunioni interne;
- **Norme di Progetto (NdP):** descrizione dei processi adottati e dell'organizzazione interna;
- **Piano di Progetto (PdP):** pianificazione delle attività, disponibilità delle risorse, avanzamento del lavoro;
- **Analisi dei Requisiti (AdR):** raccolta e formalizzazione dei requisiti della proponente;
- **Piano di Qualifica (PdQ):** criteri di verifica e validazione, test effettuati e risultati;
- **Glossario:** definizione dei termini tecnici e concetti chiave utilizzati nel progetto;
- **Dichiarazione degli Impegni:** stima dei costi del progetto, ore per ruolo e responsabilità dei componenti del gruppo;
- **Lettera di Presentazione:** presentazione formale del gruppo, fornisce una panoramica delle risorse consumate, rimanenti e metodologie usate;
- **Valutazione dei Capitolati:** analisi dei capitolati disponibili, punti di forza, criticità e motivazioni della scelta effettuata dal gruppo.

2.2.6) Strumenti a supporto

Per svolgere le attività del processo di fornitura, il gruppo utilizza strumenti sia interni sia esterni:

Strumenti interni

- **GitHub**: gestione backlog, ticketing, pubblicazione dei documenti;
- **Google Calendar**: gestione appuntamenti e scadenze;
- **Discord / Whatsapp**: coordinamento interno e riunioni del gruppo.

Strumenti verso la proponente

- **Google Mail**: comunicazioni ufficiali scritte;
- **Zoom**: riunioni sincrone remote;
- **Telegram**: indicata dalla proponente per chiarimenti rapidi.

2.3) Processo di sviluppo

2.3.1) Introduzione

Il processo di sviluppo descrive l'insieme delle attività necessarie alla realizzazione del prodotto software, a partire dall'analisi dei requisiti fino alla consegna e accettazione del sistema finale.

Secondo lo [Standard ISO 12207:2017](#), il processo di sviluppo comprende tutte le attività tecniche volte a trasformare i requisiti concordati con il committente in un **prodotto software funzionante, verificato e conforme agli obiettivi di qualità stabiliti**. Tali attività includono l'analisi dei requisiti, la progettazione dell'architettura, la codifica, l'integrazione, il testing e la validazione del sistema.

Nel contesto del progetto, il processo di sviluppo è adottato per garantire che ogni fase di realizzazione del software sia svolta in modo **sistematico, tracciabile e coerente** con le specifiche definite, assicurando la **qualità, l'affidabilità e la manutenibilità** del prodotto nel tempo.

2.3.2) Scopo del processo

Lo scopo del processo di sviluppo è garantire la corretta realizzazione del prodotto software in conformità ai requisiti funzionali, di qualità e di vincolo definiti nel capitolato e nell'Analisi dei Requisiti.

In particolare, il processo di sviluppo ha l'obiettivo di:

- **Trasformare i requisiti approvati in una soluzione software** progettata e implementata correttamente;
- **Assicurare la tracciabilità** tra requisiti, componenti progettuali, codice e test;
- Garantire che il software **soddisfi gli standard di qualità** stabiliti;
- Individuare e correggere tempestivamente eventuali difetti attraverso **attività di verifica e validazione**;

- Produrre un sistema software **affidabile, manutenibile e conforme alle aspettative** della proponente.

Il processo di sviluppo costituisce quindi il fulcro tecnico del progetto e guida in modo strutturato tutte le attività necessarie alla costruzione del prodotto finale.

2.3.3) Attività del processo

Il processo di sviluppo è articolato in un insieme di attività tra loro correlate, definite in conformità allo **standard ISO/IEC 12207**, che guidano la realizzazione del prodotto software lungo l'intero ciclo di vita.

Le principali attività previste sono le seguenti:

1. Raccolta e Analisi dei Requisiti

Attività volta all'identificazione, analisi e formalizzazione dei requisiti funzionali, di qualità e di vincolo del sistema. I requisiti vengono raccolti a partire dal capitolato e dal confronto con la proponente, documentati nell'Analisi dei Requisiti e resi tracciabili per le successive fasi di progettazione, implementazione e verifica.

2. Progettazione dell'architettura

Definizione della struttura generale del sistema, individuando le componenti principali, le loro responsabilità e le interazioni tra di esse, al fine di soddisfare i requisiti individuati.

3. Progettazione di dettaglio

Definizione dettagliata delle singole componenti software e delle unità che le compongono, fornendo le informazioni necessarie alla fase di codifica.

4. Codifica

Implementazione del software secondo quanto definito in fase di progettazione, adottando convenzioni di stile e buone pratiche di programmazione per garantire leggibilità, manutenibilità e qualità del codice.

5. Test e Integrazione

Verifica del corretto funzionamento tramite i test di sistema e successiva integrazione delle componenti, accompagnata da test di integrazione per individuare eventuali difetti.

6. Rilascio e Distribuzione

Consegna del prodotto software nelle modalità concordate e supporto alla proponente nelle attività di accettazione, al fine di verificare il soddisfacimento dei requisiti contrattuali.

2.3.4) Inquadramento del processo nelle Baseline di progetto

Il processo di sviluppo è strettamente collegato alle baseline previste dal progetto:

Requirements and Technology Baseline (RTB)

Comprende principalmente le attività di:

- Analisi dei Requisiti;
- Prime attività di codifica e prototipazione nella forma di Proof of Concept.

Product Baseline (PB)

Comprende principalmente le attività di:

- Progettazione dell'architettura software;
- Codifica completa del prodotto.

2.3.5) Procedure operative

Le attività del processo di sviluppo vengono svolte seguendo procedure operative standardizzate, al fine di garantire coerenza, tracciabilità e qualità del prodotto software.

Ogni procedura è supportata e documentata tramite i principali artefatti di progetto: Analisi dei Requisiti (AdR), Piano di Progetto (PdP), Piano di Qualifica (PdQ) e Norme di Progetto (NdP).

Analisi dei requisiti

- Analizzare il capitolato e la documentazione fornita dalla proponente;
- Raccogliere eventuali chiarimenti tramite comunicazioni ufficiali;
- Identificare gli attori del sistema e le principali interazioni con il software;
- Individuare e descrivere i casi d'uso, specificando per ciascuno
 1. attori coinvolti;
 2. scenario principale;
 3. scenari alternativi ed eccezioni;
- Assegnare a ogni caso d'uso un identificativo univoco secondo una nomenclatura coerente;
- Derivare dai casi d'uso e dal capitolato i requisiti del sistema;
- Classificare i requisiti in:
 1. requisiti funzionali;
 2. requisiti di qualità;
 3. requisiti di vincolo;
- Assegnare a ciascun requisito:
 1. un identificativo univoco;
 2. una priorità (obbligatorio, desiderabile, opzionale);
- Formalizzare casi d'uso e requisiti nel documento di Analisi dei Requisiti (AdR);
- Associare ogni requisito a uno o più casi d'uso;

- Garantire la tracciabilità dei requisiti verso le successive attività di progettazione, codifica e verifica, come riportato in AdR e PdQ.

Progettazione

- Definire l'architettura generale del sistema sulla base dei requisiti approvati;
- Individuare le componenti software e le loro responsabilità;
- Definire le interfacce tra le componenti;
- Aggiornare la documentazione di progetto in base alle decisioni architetturali;
- Verificare la coerenza tra requisiti definiti in AdR e le scelte progettuali;
- Allineare le attività progettuali alla pianificazione definita nel Piano di Progetto (PdP).

Codifica

- Implementare le funzionalità secondo la progettazione approvata;
- Applicare le convenzioni di stile e le buone pratiche definite nelle Norme di Progetto;
- Utilizzare il sistema di versionamento per la gestione del codice;
- Suddividere il lavoro in attività pianificate nel PdP e tracciate tramite issue;
- Eseguire controlli statici, formattazione automatica e verifiche preliminari del codice;
- Garantire la tracciabilità tra codice implementato e requisiti definiti in AdR.

Verifica e test

- Definire i casi di test in conformità al Piano di Qualifica (PdQ);
- Eseguire test di unità sulle singole componenti software;
- Eseguire test di integrazione tra le componenti;
- Registrare l'esito dei test, le non conformità e le eventuali correzioni nel PdQ;
- Verificare la copertura dei requisiti definiti in AdR attraverso i test eseguiti.

Integrazione e rilascio

- Integrare progressivamente le componenti software secondo la pianificazione del PdP;
- Verificare il corretto funzionamento del sistema nel suo insieme;
- Eseguire i test di qualifica di sistema previsti dal PdQ;
- Preparare il materiale di rilascio e la documentazione associata;
- Effettuare la consegna secondo le modalità e le scadenze concordate con la proponente.

Gestione delle modifiche

- Registrare le richieste di modifica come issue;
- Valutare l'impatto delle modifiche sui requisiti (AdR) e sulla pianificazione (PdP);
- Aggiornare la documentazione di progetto e il codice;
- Eseguire nuovamente le verifiche previste dal PdQ;
- Garantire il mantenimento della tracciabilità tra requisiti, codice e test.

2.3.5.1) Documenti principali

Durante il processo di sviluppo, le attività sono supportate dalla documentazione di progetto già definita nel processo di fornitura (AdR, PdP, PdQ, NdP), con l'aggiunta di riferimenti specifici alle fasi di implementazione e verifica software.

2.3.6) Progettazione

2.3.6.1) Descrizione e scopo

La progettazione rappresenta l'attività che trasforma i requisiti definiti nell'Analisi dei Requisiti in una soluzione software strutturata, definendo l'architettura del sistema e le specifiche tecniche necessarie alla fase di implementazione.

Secondo lo standard ISO/IEC 12207, la progettazione comprende:

- La **progettazione architetturale**, che definisce la struttura generale del sistema, le componenti principali e le loro interazioni;
- La **progettazione di dettaglio**, che specifica il comportamento interno di ciascuna componente software.

Lo scopo della progettazione è:

- **Definire una soluzione tecnica** che soddisfi i requisiti funzionali, di qualità e di vincolo specificati nell'Analisi dei Requisiti;
- **Garantire modularità, riusabilità e manutenibilità** del sistema attraverso una corretta suddivisione in componenti;
- **Facilitare la fase di codifica** fornendo specifiche chiare e complete delle unità software da implementare;
- **Assicurare la tracciabilità** tra requisiti, componenti architetturali e implementazione;
- **Documentare le scelte progettuali** per consentire la verifica della conformità del prodotto rispetto alle specifiche e supportare la manutenzione futura.

Le decisioni architetturali e le scelte tecniche specifiche adottate dal gruppo sono documentate nella Specifica Tecnica, che costituisce il riferimento progettuale per l'implementazione del sistema.

2.3.6.2) Standard e linee guida da seguire

La progettazione del sistema deve conformarsi agli standard UML e ai design pattern consolidati presentati nel corso di Ingegneria del Software.

2.3.6.2.1) Diagrammi UML

Per la rappresentazione della progettazione si adottano le convenzioni UML definite nelle slide del corso:

- **Diagrammi delle classi** : per la struttura statica del sistema, le relazioni tra classi e le dipendenze;
- **Diagrammi di sequenza**: per le interazioni dinamiche tra componenti durante l'esecuzione di casi d'uso;
- **Diagrammi dei package**: per l'organizzazione architetturale ad alto livello.

2.3.6.2.2) Design Pattern

I design pattern da considerare durante la progettazione sono documentati nelle slide del corso:

- **Pattern architetturali** : per l'organizzazione dell'architettura e la gestione delle dipendenze;
- **Pattern comportamentali** : per la definizione della comunicazione tra oggetti.

I pattern concretamente applicati nel progetto sono documentati nella Specifica Tecnica (Sezione Design Patterns).

2.3.6.2.3) Principi di progettazione

La progettazione segue i seguenti principi fondamentali:

- **Separazione delle responsabilità**: ogni componente ha una responsabilità ben definita;
- **Modularità**: organizzazione in moduli indipendenti e riutilizzabili;
- **Information hiding**: dettagli implementativi nascosti dietro interfacce;
- **Basso accoppiamento**: minimizzare le dipendenze tra componenti;
- **Alta coesione**: raggruppare funzionalità correlate.

2.3.6.3) Strumenti per la progettazione

Per la creazione e manutenzione dei diagrammi di progettazione, il gruppo utilizza i seguenti strumenti:

- **PlantUML**: per la generazione programmatica dei diagrammi UML (classi, sequenza, package). Questo approccio diagram-as-code permette di concentrarsi esclusivamente sulla logica e sui contenuti. Inoltre, la possibilità di organizzare i modelli in file separati e di sfruttare le direttive di inclusione assicura un'elevata coerenza strutturale, ottimizzando i tempi di produzione e semplificandone la manutenibilità.
- **Draw.io**: utilizzato per la creazione di diagrammi visivamente complessi che necessitano di una resa grafica specifica o del posizionamento manuale degli elementi, superando i vincoli di layout di PlantUML.

2.3.6.4) Procedure di progettazione

2.3.6.4.1) Documentazione delle scelte progettuali

Le decisioni architettureali e le scelte progettuali devono essere documentate nella Specifica Tecnica, che costituisce il riferimento ufficiale per l'implementazione.

La Specifica Tecnica include:

- Descrizione dell'architettura di sistema
- Design pattern applicati
- Diagrammi delle classi dettagliati
- Schema dei dati e strutture di persistenza

2.3.6.4.2) Aggiornamento della documentazione

La documentazione di progetto deve essere mantenuta aggiornata durante l'evoluzione del sistema:

- Aggiornare la Specifica Tecnica in seguito a modifiche architettureali significative;
- Documentare le decisioni progettuali nei verbali o nelle issue quando rappresentano cambiamenti rilevanti;
- Mantenere la coerenza tra Specifica Tecnica, Analisi dei Requisiti e codice implementato.

2.3.7) Codifica

2.3.7.1) Descrizione e scopo

La codifica rappresenta la fase di implementazione del software secondo quanto definito in fase di progettazione. Durante questa attività, i programmatori traducono le specifiche architettureali e di dettaglio in codice sorgente eseguibile.

Lo scopo della codifica è:

- **Implementare le funzionalità** definite nell'Analisi dei Requisiti e nella Specifica Tecnica;
- **Garantire la qualità del codice** attraverso l'adozione di convenzioni di stile, buone pratiche di programmazione e strumenti di analisi automatica;
- **Assicurare la manutenibilità** del software mediante codice leggibile, ben strutturato e adeguatamente documentato;
- **Facilitare la testabilità** scrivendo codice modulare e disaccoppiato;

La codifica deve seguire le convenzioni e le norme definite in questo documento per garantire uniformità e coerenza nell'intero progetto.

2.3.8) Strumenti a supporto

Per lo sviluppo del software, il gruppo utilizza strumenti mirati a garantire qualità, tracciabilità e collaborazione:

- **Linguaggio e ambiente di sviluppo:** Python 3.x come linguaggio principale per le componenti software del sistema backend, Vue 3.X per il frontend.
- **Versionamento del codice:** Git/GitHub per gestione dei repository, branch, commit, issue e pull request.
- **Formattazione e controllo del codice:** Il progetto adotta controlli automatici di qualità del codice e della documentazione tramite pipeline CI, includendo formattazione, verifica di leggibilità (indice Gulpease) e generazione automatica della documentazione.
- **Gestione attività e tracciamento:** GitHub Issues per assegnazione, monitoraggio e gestione delle modifiche.
- **Comunicazione e collaborazione interna:** Discord o WhatsApp per coordinamento rapido, aggiornamenti sullo stato di avanzamento e chiarimenti tra membri del gruppo.
- **Comunicazione verso la proponente:** email ufficiale, Zoom per riunioni sincrone e Telegram per chiarimenti rapidi.

2.3.9) Convenzioni di stile

Per garantire di qualità dello sviluppo del codice verranno usati i seguenti criteri (i nomi delle classi e delle variabili saranno in inglese):

- **Backend :**
 - **Variabili, attributi, funzioni e metodi:** snake_case.
 - **Classi, Data Transfer Object (DTO), Interfacce (Porte):** PascalCase, nel caso delle interfacce si aggiunge il prefisso Interface (es. InterfaceDeviceRepository).
 - **Costanti a livello di Modulo:** UPPER_SNAKE_CASE.
- **Frontend :**
 - **Variabili, attributi, funzioni e metodi:** camelCase.
 - **Handler di eventi:** camelCase con prefisso handle (es. handleSubmit()).
 - **Componenti Vue:** PascalCase.
- **Persistenza (MongoDB) :**
 - **Campi dei documenti:** snake_case (es.device_name).
 - **Nomi delle collection:** lowercase al plurale (es. devices, modules).
- **Indentazioni :**

I blocchi annidati del codice devono seguire un'indentazione equivalente a 2 spazi, sia nel Backend che nel Frontend.

2.3.9.1) Buone pratiche di programmazione

2.3.9.1.1) Principi di qualità del codice

Ogni componente del gruppo deve attenersi ai seguenti principi per garantire codice di qualità:

- **Riuso del codice:** sviluppare funzioni e metodi modulari che possano essere riutilizzati in diverse parti del progetto, evitando la duplicazione del codice e facilitando la manutenzione;
- **Manutenibilità:** scrivere codice chiaro, leggibile e ben strutturato. I nomi di variabili, funzioni e classi devono essere significativi e auto-esplicativi. Il codice deve essere organizzato logicamente e seguire i principi di separazione delle responsabilità;
- **Efficienza:** prestare attenzione alla complessità algoritmica e alle performance, cercando di evitare soluzioni inefficienti quando esistono alternative più performanti;
- **Testabilità:** progettare il codice in modo che sia facilmente testabile, favorendo il disaccoppiamento delle dipendenze e l'uso di interfacce.

2.3.9.1.2) Struttura del codice

Lunghezza e complessità:

- Preferire funzioni brevi e ben definite che svolgano un singolo compito;
- Evitare funzioni eccessivamente lunghe o con logica complessa;
- Suddividere funzioni complesse in sotto-funzioni più semplici e riutilizzabili;

Variabili globali: evitare l'uso di variabili globali dove possibile, preferendo il passaggio esplicito di parametri e l'uso di dependency injection.

Parametri di funzione: gli inbound adapter costruiscono dei command object da passare ai service, in questo modo la firma rimane pulita e invariata indipendentemente dal numero di parametri.

2.3.9.1.3) Gestione degli errori

Backend (Python):

- Utilizzare blocchi try-except per gestire eccezioni previste;
- Definire custom exceptions per errori specifici del dominio applicativo;
- Loggare errori significativi per facilitare il debugging;
- Non catturare eccezioni generiche (except Exception) se non strettamente necessario; preferire eccezioni specifiche.

Frontend (JavaScript/Vue.js):

- Utilizzare blocchi try-catch per gestire errori nelle chiamate asincrone e nelle operazioni critiche;
- Fornire feedback all'utente in caso di errori (messaggi, notifiche);
- Loggare errori in console per facilitare il debugging in fase di sviluppo.

2.3.9.2) Formattazione e controllo qualità del codice

Il progetto adotta controlli automatici di qualità del codice attraverso strumenti di formattazione, analisi statica e pipeline CI/CD.

2.3.9.2.1) Strumenti per il backend (Python)

Ruff ($\geq 0.15.12$): linter e formatter per Python che sostituisce Ruff viene utilizzato per:

- Formattazione automatica del codice secondo lo stile PEP 8;
- Analisi statica per identificare errori comuni, code smells e violazioni di stile;
- Ordinamento automatico degli import.

MyPy ($\geq 1.20.2$): type checker statico per Python, utilizzato per verificare la correttezza dei type hints e identificare errori di tipo a tempo di sviluppo.

Pytest con coverage:

- `pytest-cov` ($\geq 7.1.0$): per misurare la copertura del codice da parte dei test;
- Configurato per generare report in formato HTML, XML e JSON;
- Target di copertura: branch coverage incluso.

2.3.9.2.2) Strumenti per il frontend (js + Vue)

Vue(v3.5.32): framework progressivo principale utilizzato per la creazione dell'interfaccia utente e dei componenti.

Pinia(v3.0.4): Lo standard per la gestione dello stato globale in Vue 3, utilizzato nel tuo progetto per gestire l'albero decisionale, le valutazioni e la sincronizzazione della UI.

D3 (v7.9.0): Libreria per la manipolazione di documenti basati sui dati, essenziale per il rendering e il layout del canvas SVG dell'albero decisionale

Vite (v8.0.9): Il bundler e dev-server che orchestra la compilazione del frontend.

@Vitejs/plugin-vue (v6.0.6): Il plugin ufficiale che permette a Vite di comprendere e compilare correttamente i file Single-File Component (SFC) con estensione `.vue`.

2.3.9.2.3) Automazione tramite CI/CD

Gli strumenti di qualità del codice sono integrati nella pipeline CI/CD tramite GitHub Actions, che esegue automaticamente:

- Formattazione del codice con Ruff;
- Analisi statica del codice (Ruff, MyPy);
- Esecuzione della suite di test con report di copertura;
- Verifica della complessità del codice;
- Generazione di report HTML per la visualizzazione dei risultati.

La pipeline di CI/CD blocca il merge di Pull Request che:

- Non rispettano gli standard di formattazione;
- Contengono errori rilevati dall'analisi statica;
- Non superano i test definiti;
- Riducono significativamente la copertura del codice.

2.3.9.2.4) Esecuzione locale

Gli sviluppatori devono eseguire i controlli di qualità in locale prima di effettuare commit e push:

```
# Formattazione e linting
poetry run ruff check . --fix
poetry run ruff format .
```

```
# Type checking
poetry run mypy backend/src
```

```
# Test con copertura
poetry run pytest
```

```
#Test relativi al frontend
npx vitest run
```

3) Processi di Supporto

3.1) Introduzione ai processi di supporto

I processi di supporto hanno lo scopo di garantire **l'efficace gestione, controllo e qualità delle attività del ciclo di vita del progetto**, fornendo strumenti, procedure e linee guida per supportare i processi primari (analisi, progettazione, sviluppo, verifica).

In conformità agli standard ISO di riferimento, i processi di supporto considerati nel presente documento includono:

1. **Documentazione:**

Processo volto a registrare, organizzare e rendere disponibili tutte le informazioni prodotte durante il ciclo di vita del progetto, assicurando tracciabilità, trasparenza e coerenza dei contenuti.

2. **Gestione delle configurazioni:**

Processo finalizzato a identificare, controllare, monitorare e aggiornare tutti gli elementi del progetto, garantendo la corretta gestione delle versioni e delle modifiche.

3. **Accertamento qualità:**

Processo che comprende attività sistematiche di verifica volte a controllare che processi, documenti e prodotti intermedi siano conformi ai requisiti, agli standard e alle metriche di qualità definiti durante lo svolgimento del progetto.

4. **Processo di Qualifica:**

Processo che comprende attività di verifica e validazione finalizzate a stabilire se un prodotto o deliverable può essere considerato concluso e idoneo al rilascio, garantendo la conformità alle specifiche tecniche e funzionali.

3.2) Processo di documentazione

3.2.1) Introduzione

Il processo di documentazione definisce le **modalità con cui il team raccoglie, organizza e gestisce i documenti** prodotti durante il progetto.

Fornisce un quadro chiaro dei **flussi documentali, delle responsabilità e degli strumenti utilizzati**, garantendo che le informazioni siano aggiornate, accessibili e coerenti con gli obiettivi del progetto.

3.2.2) Scopo del processo

Il processo di documentazione ha lo scopo di **registrare, organizzare e rendere disponibili tutte le informazioni** prodotte durante il ciclo di vita del progetto.

Esso garantisce:

- **Tracciabilità** delle decisioni;
- **Coerenza** tra le attività svolte;
- **Trasparenza** verso gli stakeholder e supporto del lavoro collaborativo del team.

La documentazione prodotta costituisce un **riferimento chiaro e accessibile** per tutte le fasi del progetto, riducendo ambiguità e fraintendimenti.

3.2.3) Attività del processo

3.2.3.1) Impegno del gruppo

Il gruppo si impegna a fornire alla proponente e ai docenti tutta la documentazione necessaria a supportare le attività di analisi, progettazione, sviluppo e verifica del progetto. Tale documentazione ha lo scopo di garantire **trasparenza, tracciabilità e qualità del lavoro** svolto, oltre a costituire un **riferimento chiaro per tutti gli stakeholder** coinvolti.

3.2.3.2) Attività principali

Le principali attività che compongono questo processo sono:

1. **Identificazione dei documenti**

Individuazione dei documenti necessari e definizione delle relative responsabilità.

2. **Progettazione dei documenti**

Applicazione delle norme e del workflow stabilito.

3. **Pubblicazione e Distribuzione della Documentazione**

Generazione del documento nel formato previsto e distribuzione ai destinatari autorizzati.

4. **Manutenzione**

Aggiornamento continuo dei contenuti e gestione delle modifiche e delle revisioni.

5. **Archiviazione e tracciabilità**

Gestione del versionamento, conservazione dei documenti e garanzia dell'accessibilità nel tempo.

3.2.4) Procedure operative

3.2.4.1) Identificazione dei documenti

Si tratta di un'attività di pianificazione in cui ogni documento viene definito secondo le seguenti caratteristiche principali:

- Titolo;
- Scopo;
- Destinatari;
- Procedure e responsabilità nella redazione e gestione del documento;
- Pianificazione per le versioni e i loro contenuti.

Per la redazione dei documenti è corretto e necessario fare riferimento a fonti autorevoli e aggiornate, quali standard ISO, università e organizzazioni ufficialmente

riconosciute, materiale didattico e link di approfondimento forniti durante il corso. Eventuali informazioni provenienti da altre fonti, se ritenute utili, devono essere obbligatoriamente verificate per accertarne la correttezza e l'affidabilità.

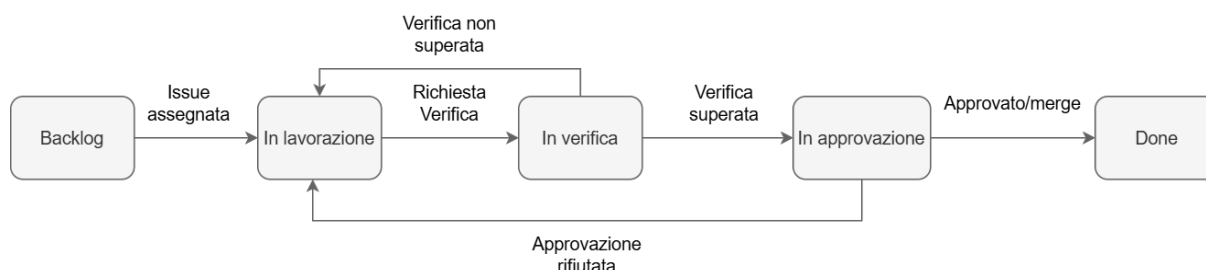
3.2.4.2) Progettazione dei documenti

Ogni documento identificato all'interno dello sviluppo software deve rispettare alcuni standard di documentazione uguali per tutti:

- Essere in formato A4;
- I contenuti inseriti devono essere coerenti con lo scopo del documento stesso;
- Tutti i documenti devono includere un indice dei contenuti e delle relative sotto-sezioni visibile all'inizio (ad eccezione del diario di bordo, che ne è esentato);
- Ogni pagina deve contenere nel header e nel footer:
 1. La sezione corrente del documento (in alto a sinistra);
 2. Il nome del gruppo (in alto a destra);
 3. Il titolo del documento (in basso a sinistra);
 4. Il numero della pagina : espresso in numeri romani per la prefazione e in numeri arabi per le pagine del corpo del documento.

3.2.4.3) Workflow documentale

All'interno dell'ambito documentale si è optato per il seguente modello per descrivere e modellare le attività necessarie a produrre un documento:



3.2.4.4) Stati del documento

- **Backlog**: magazzino delle attività da svolgere, ogni documento inizia in questo stato.
- **In lavorazione**: il documento è stato preso in carico dal componente del gruppo nel ruolo previsto.
- **In verifica**: Il lavoro dell'autore è terminato. Il documento deve ora essere revisionato oppure corretto, nel caso in cui non sia stato approvato durante la fase di validazione.
- **In approvazione**: il lavoro del revisore è finito. Il documento va valutato per l'approvazione oppure respinto, fornendo le opportune motivazioni accompagnate da un elenco delle correzioni da apportare.
- **Done**: il documento è stato approvato.

3.2.4.5) Procedura di avanzamento tra stati

- Da **Backlog** a **In lavorazione**: l'Amministratore crea la issue predisponendo l'ambiente tecnico; contestualmente, in accordo con il Responsabile, viene determinato e impostato l'assegnatario (componente del gruppo nel ruolo pertinente) che prende in carico l'attività.
- Da **In lavorazione** a **In verifica**: l'assegnatario dichiara conclusa la propria attività, trasferendo la issue in revisione e assegnandola al Verificatore (deciso a priori) che verrà notificato automaticamente.
- Da **In verifica** a **In lavorazione** (Rifiuto del Verificatore): il Verificatore rileva che non è conforme e riassegna la issue all'esecutore con un report delle anomalie da sanare.
- Da **In verifica** a **In approvazione**: il Verificatore accerta la correttezza dell'incremento e sposta la issue in approvazione, assegnandola al Responsabile per il controllo finale di coerenza.
- Da **In approvazione** a **In lavorazione** (Rifiuto del Responsabile): il Responsabile rileva incongruenze di alto livello o mancanze strategiche. La issue viene riassegnata all'assegnatario originale per la correzione, informando il Verificatore della svista.
- Da **In approvazione** a **Done**: il Responsabile accetta la revisione proposta e chiude la issue con

```
git commit -m "commento. Close #numero_issue"
```

3.2.4.6) Procedura di archiviazione e tracciabilità

I documenti sono salvati sull'apposito repository.

Il path relativo è ricavabile nel seguente modo:

(Fanno eccezione i diari di bordo, in quanto fanno parte delle regole di progetto, ma non supportano alcun processo primario, perciò sono salvati nella cartella **./src/DiariDiBordo**)

./<Type>/<Baseline>/<Destinatari>/<Cartella del Documento>

Type:

- **src** per i file in formato typst.
- **output** per i pdf.

Baseline:

- **RTB** per i documenti allo stato della RTB.
- **PB** per i documenti allo stato della PB.

Destinatari:

- DocumentazioneInterna per i documenti ad uso interno.
- DocumentazioneEsterna per i documenti ad uso esterno.

Cartella del Documento:

- VerbalInterni.

- VerbalEsterni.
- Per documenti complessi coincide con il nome del documento².

3.2.4.7) Pubblicazione e Distribuzione della Documentazione

I documenti vengono ricompilati automaticamente in PDF tramite GitHub Action e sono consultabili da tutti i membri del team. Sono disponibili nella repository del gruppo dedicata alla [Documentazione](#). La posizione ufficiale dei file è indicata nel README.md del repository.

Per facilitarne la consultazione è inoltre possibile accedere al [sito web ufficiale](#) del gruppo, creato appositamente per visualizzare e navigare i documenti in modo più immediato.

La versione del documento e la tracciabilità delle modifiche sono gestite tramite il Registro delle Modifiche, integrato direttamente all'interno di ciascun documento.

3.2.5) Inquadramento del processo nelle Baseline di progetto

La documentazione fornita in corrispondenza della fase RTB (Requirements and Technology Baseline) comprende sia materiali tecnici sia documenti operativi e organizzativi, al fine di garantire una valutazione completa e trasparente dello stato del progetto.

In particolare, vengono prodotti:

Documenti tecnici :

- Analisi dei Requisiti (AdR);
- Piano di Progetto (PdP);
- Piano di Qualifica (PdQ);
- Preventivo dei Costi e delle risorse impiegate;

Documenti operativi e organizzativi

- Norme di Progetto (NdP);
- Verbali interni;
- Verbali esterni;
- Glossario;
- Diario di bordo.

3.2.6) Documentazione fornita

3.2.6.1) Elenco dei documenti

Documento 1	Analisi dei Requisiti	23
Documento 2	Piano di Qualifica	23

²Per motivi di manutenibilità e facilità di aggiornamento i contenuti del file typst sono stati divisi in più file quando una loro sezione diventa eccessivamente corposa.

Documento 3 Piano di Progetto	24
Documento 4 Verballi	25
Documento 5 Diario di Bordo	26
Documento 6 Norme di Progetto	26
Documento 7 Glossario	27
Documento 8 Specifica Tecnica	27
Documento 9 Manuale Utente	28

3.2.6.2) Informazioni comuni

Ogni documento presenta una sezione iniziale standardizzata per tutti i membri del team. Questa sezione viene generata utilizzando un apposito template centrale e unico, al fine di garantire coerenza e facilitare la compilazione.

La sezione iniziale è composta dai seguenti elementi:

- 1. Pagina di copertina** : contenente il titolo del documento, il nome e il logo del gruppo e le relative informazioni di contatto.
- 2. Tabella dello stato** : che riassume lo stato del documento e informazioni generali quali versione, autori, verificatori, uso e destinatari.
- 3. Registro delle modifiche** : costituito da una tabella contenente le informazioni sul versionamento e sulla tracciabilità.
- 4. Indice dei contenuti** : aggiornato automaticamente tramite sintassi Typst.
- 5. Indice delle immagini e delle tabelle** : presente solo nei documenti che ne contengono.

3.2.6.3) Struttura specifica

Di seguito viene riportata la struttura standard dei documenti principali, le rispettive sezioni, il loro scopo, i destinatari, e le metodologie adottate per la scrittura e la revisione, al fine di mantenere coerenza e uniformità all'interno del gruppo.

3.2.6.4) Analisi dei Requisiti

L'Analisi dei Requisiti ha il compito di descrivere in modo completo, chiaro e verificabile tutte le **funzionalità** che il sistema deve offrire, includendo sia **requisiti funzionali** sia **non funzionali**.

Il documento fornisce inoltre i principali **casi d'uso, con attori e scenari associati**, e garantisce la **tracciabilità tra requisiti**, casi d'uso ed eventuali **estensioni future**. Rappresenta un riferimento stabile per sviluppatori, tester e manutentori durante tutte le fasi del progetto.

Destinatari : stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.2.6.4.1) Struttura principale

Il documento comprende:

- Definizione formale dei requisiti funzionali e non funzionali;
- Modellazione dei casi d'uso con attori e flussi narrativi;
- Matrice di tracciabilità requisiti-casi d'uso;
- Eventuali vincoli tecnici, operativi o di contesto.

Documento 1: Analisi dei Requisiti

3.2.6.5) Piano di Qualifica

Il Piano di Qualifica ha l'obiettivo di garantire che il prodotto sviluppato rispetti elevati standard di qualità. Definisce processi, metriche, strategie di testing e criteri di valutazione necessari a verificare la qualità del software e del processo di sviluppo. Fornisce inoltre strumenti operativi per la misurazione e la validazione dei risultati.

Destinatari: stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.2.6.5.1) Struttura principale

Il documento è articolato nelle seguenti componenti:

- Metriche di qualità del prodotto e del processo;
- Strategie, livelli e tecniche di testing;
- Piano delle verifiche e validazioni;
- Cruscotto qualità con indicatori e soglie di accettazione.

Documento 2: Piano di Qualifica

3.2.6.6) Piano di Progetto

Il Piano di Progetto definisce la pianificazione complessiva delle attività, descrivendo l'approccio adottato dal gruppo.

Il documento fornisce una visione strutturata dell'**organizzazione del lavoro**, includendo la definizione degli obiettivi, la gestione delle risorse, l'assegnazione dei ruoli, la pianificazione temporale e l'analisi dei rischi.

La sua funzione principale è garantire un monitoraggio costante dell'**avanzamento** del progetto attraverso **revisioni periodiche** e **rendicontazioni** relative ai vari sprint.

Tale monitoraggio consente al gruppo di valutare l'efficienza del workflow, individuare tempestivamente eventuali criticità e adattare la pianificazione quando necessario.

Destinatari : stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.2.6.6.1) Struttura principale

La struttura del documento comprende:

- Ambito e obiettivi del progetto;
- Analisi dei rischi e piani di mitigazione;
- Preventivo iniziale e disponibilità delle risorse;
- Pianificazione di lungo periodo;
- Sezione dedicata alle revisioni, che per ogni sprint comprende:
 - **Attività pianificate** → obiettivi e task previsti per il periodo di sprint;
 - **Gestione dei rischi** → analisi proattiva dei rischi attesi e dei rischi emersi, con indicazione delle relative strategie di mitigazione e piani di contingenza;
 - **Preventivo ore per ruolo** → stima dell'effort pianificato, suddiviso per responsabilità;
 - **Retrospektiva del gruppo** → riflessioni su apprendimento, workflow ed efficacia della collaborazione;
 - **Consuntivo ore effettive** → rendicontazione delle ore realmente impiegate dal gruppo nello sprint per aggiornare il budget residuo.

Documento 3: Piano di Progetto

3.2.6.7) Verbali

I verbali sono suddivisi in due categorie principali :

- Verbali interni -> documentano riflessioni e confronti avvenuti esclusivamente tra i membri del gruppo.
- Verbali esterni -> vengono redatti in corrispondenza di riunioni o confronti con l'azienda di riferimento (Bluewind).

Ogni verbale si conclude con una **riflessione finale del gruppo**, dalla quale emergono decisioni operative che vengono successivamente formalizzate tramite la creazione di **issue GitHub**, che il gruppo si impegna a completare.

3.2.6.7.1) Struttura principale

Ogni verbale deve avere la seguente suddivisione numerata:

1. **Informazioni comuni della Sezione 3.2.6.2** (standard condivisi di documento)
2. **Informazioni generali**
 - Data e luogo della riunione;
 - Orario di inizio/fine;
 - Partecipanti;
 - Tipologia (interno/esterno);
 - Motivo (principalmente per verbali esterni);
 - Scriba (responsabile del verbale in quel momento).
3. **Ordine del giorno** : scaletta dei temi da discutere, raccolti e organizzati del responsabile sulla base dei contributi dei membri del gruppo o dei referenti aziendali.
4. **Riassunto della riunione** : sintesi breve e oggettiva dei punti discussi.
5. **Decisioni** : azioni o obiettivi (anche ad alto livello) che il gruppo deve intraprendere per dare seguito alla riunione.
6. **TODO** : attività specifiche derivate dalle decisioni. Una singola decisione può essere suddivisa in più TODO, che complessivamente consentono di raggiungere l'obiettivo stabilito.

Documento 4: Verbali

3.2.6.8) Diario di Bordo

Il Diario di Bordo è un'attività prevista dal Prof. Tullio Vardanega all'interno del progetto di Ingegneria del Software. Rappresenta un momento di condivisione in cui ciascun gruppo espone il proprio stato di avanzamento, con particolare attenzione a dubbi o problematiche emerse durante lo svolgimento delle attività.

Composto principalmente da:

Titolo: Diario di Bordo seguito dal numero progressivo associato.

Scopo: Fornire ai gruppi un feedback sulle attività svolte e consentire di portare all'attenzione comune eventuali dubbi relativi al processo di lavoro.

Destinatari: Prof. Tullio Vardanega e gli altri gruppi coinvolti nel progetto.

Documento 5: Diario di Bordo

3.2.6.9) Norme di Progetto

Le Norme di Progetto definiscono l'insieme di regole, convenzioni e standard adottati dal gruppo al fine di garantire coerenza, qualità e uniformità nella produzione della documentazione, del codice e dei deliverable. Il documento stabilisce inoltre procedure condivise per redazione, revisione, versionamento, gestione dei file e comunicazione interne, riducendo il rischio di errori, fraintendimenti o incoerenze operative tra i membri del team.

Destinatari : Tutti i membri del gruppo di progetto (interno)

3.2.6.9.1) Struttura principale

Il documento delle Norme di Progetto è organizzato secondo i tipi di processo presenti nel progetto:

- **Processi primari:** attività direttamente legate alla realizzazione del prodotto software.
- **Processi di supporto:** attività che garantiscono qualità, tracciabilità e gestione della documentazione.
- **Processi organizzativi:** attività relative alla gestione del team, pianificazione e coordinamento.

Documento 6: Norme di Progetto

3.2.6.10) Glossario

Il Glossario ha l'obiettivo di garantire **chiarezza e uniformità nella terminologia** utilizzata nei documenti di progetto, sia verso l'esterno (proponente, docenti) sia all'interno del gruppo di lavoro.

Raccoglie i **termini ritenuti non banali, abbreviazioni e acronimi**, fornendo definizioni precise per ridurre ambiguità interpretative e favorire una comprensione condivisa dei concetti chiave.

Il documento è soggetto ad **aggiornamento continuo**, in modo da riflettere l'evoluzione del progetto e mantenere allineata la terminologia. Oltre alla versione ufficiale inclusa nei documenti di progetto, il team mantiene un glossario interno aggiornato disponibile sul sito web della documentazione, che funge da riferimento centralizzato sempre accessibile.

Per **favorire la tracciabilità e la consultazione immediata**, all'interno dei singoli documenti tutte le parole presenti nel glossario vengono evidenziate. Questa evidenziazione consente agli utenti di riconoscere rapidamente i termini definiti formalmente e di rimandare al glossario in caso di dubbi o possibili ambiguità.

Documento 7: Glossario

3.2.6.11) Specifica Tecnica

La Specifica Tecnica ha il compito di descrivere in modo completo, chiaro e dettagliato l'architettura software e le scelte implementative del sistema.

Il documento definisce i principali componenti, i moduli, le interfacce e la logica applicativa, garantendo la tracciabilità rispetto all'analisi dei requisiti e la conformità ai vincoli progettuali. Rappresenta un riferimento stabile per sviluppatori e manutentori durante tutte le fasi di implementazione del sistema.

Destinatari : stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.2.6.11.1) Struttura principale

Il documento comprende:

- Architettura del sistema e scomposizione in moduli;
- Diagrammi delle classi e interfacce;
- Dettaglio delle tecnologie e dei pattern progettuali adottati;
- Verificabilità e tracciabilità con i requisiti.

Documento 8: Specifica Tecnica

3.2.6.12) Manuale Utente

Il Manuale Utente ha il compito di descrivere in modo chiaro, accessibile e completo le funzionalità del sistema dal punto di vista dell'utente finale.

Il documento fornisce le istruzioni per l'installazione e la configurazione iniziale, una guida passo-passo per l'utilizzo delle funzionalità principali.

Rappresenta un punto di riferimento per gli utilizzatori del software, facilitando l'apprendimento e l'adozione dello strumento.

Destinatari : utenti finali del sistema e stakeholder.

3.2.6.12.1) Struttura principale

Il documento comprende:

- Introduzione e panoramica delle funzionalità di base;
- Requisiti di sistema e istruzioni di configurazione;
- Guide operative e scenari d'uso guidati;

Documento 9: Manuale Utente

3.2.7) Strumenti di supporto

Typst: Linguaggio di markup moderno per la composizione e la tipografia di documenti, pensato come alternativa più semplice e veloce a LaTeX.

- Sintassi intuitiva;
- Supporto ad automazioni tramite template, funzioni e regole di stile riutilizzabili;
- Preview istantanea del documento.

GitHub: Strumento scelto dal gruppo per la condivisione del lavoro e la gestione delle attività tramite **issue tracking**.

- Utilizzo di GitHub Actions per la compilazione automatica dei documenti.
- Documentazione disponibile nel repository [GitHub](#).
- [Sito web](#) predisposto tramite GitHub Pages per facilitare la consultazione della documentazione.

TurboScribe AI: Per velocizzare il processo di documentazione, il gruppo utilizza il tool di intelligenza artificiale [TurboScribe](#), che consente di trascrivere registrazioni audio in formato testuale. Lo strumento è utilizzato in particolare per il supporto alla redazione dei verbali, permettendo il riascolto rapido di momenti specifici delle riunioni tramite selezione testuale.

3.3) Processo di Gestione delle Configurazioni

3.3.1) Introduzione

Il processo di Gestione delle Configurazioni ha lo scopo di garantire la corretta identificazione, controllo, monitoraggio e aggiornamento di tutti gli elementi del progetto, siano essi documenti, codice, deliverable o altri artefatti di progetto.

Questo processo assicura coerenza, tracciabilità e qualità lungo tutto il ciclo di vita del progetto, supportando le attività dei processi primari (sviluppo, fornitura) e riducendo rischi di incongruenze o perdita di informazioni.

3.3.2) Scopo del processo

Il processo di gestione delle configurazioni mira a:

1. Garantire la **tracciabilità completa delle modifiche** apportate a documenti, codice e artefatti.
2. **Controllare e gestire le versioni** in modo chiaro e uniforme.
3. Fornire un **registro delle modifiche** consultabile da tutti i membri del gruppo.
4. **Supportare la revisione, approvazione e validazione** dei deliverable.
5. **Ridurre ambiguità e conflitti** dovuti a modifiche non controllate.

3.3.3) Attività del processo

Le principali attività della Gestione delle Configurazioni includono:

1. Identificazione degli elementi di configurazione.
2. Applicazione del codice di versione X.Y.Z (stabile, feature, patch).
3. Aggiornamento automatico della versione in tutti i documenti tramite Typst/GitHub.
4. Inserimento di ogni modifica nel Registro delle Modifiche, comprensivo di: data, autore, verificatore, descrizione e nuova versione.
5. Verifica delle modifiche da parte dei revisori.
6. Approvazione e rilascio di versioni stabili.
7. Conservazione dei documenti e del codice in repository centralizzati.
8. Backup per garantire sicurezza e recuperabilità dei dati.

3.3.4) Procedure operative

3.3.4.1) Codice di versione

Ogni modifica apportata a un documento genera automaticamente una nuova versione, identificata tramite un codice nel formato:

X.Y.Z

dove ciascuna componente rappresenta uno stato diverso del processo di validazione:

- **X – Versione stabile approvata**

Indica l'ultima versione ufficialmente approvata dal Responsabile. Il suo incremento segnala una revisione sostanziale o una modifica di grande rilievo. L'incremento di X comporta l'azzeramento automatico di Y e Z.

- **Y – Feature**

Rappresenta l'ultima approvazione da parte di un Verificatore. Il suo incremento indica l'introduzione o modifica di una nuova funzionalità o sezione del documento. L'incremento di Y comporta l'azzeramento di Z.

- **Z – Patch**

Indica l'ultima modifica di dettaglio verificata (correzioni minori, refusi, aggiustamenti formali). L'incremento di Z rappresenta cambiamenti minori.

3.3.4.1.1) Regole di incremento

1. Ogni approvazione genera un incremento della cifra di versione stabile.
2. Nel versionamento X.Y.Z, maggiore è il cambiamento, più significativa è la cifra che viene incrementata: X ha un peso maggiore di Y, e Y maggiore di Z
3. L'incremento di una cifra comporta sempre l'azzeramento delle cifre alla sua destra, mantenendo coerenza nella progressione delle versioni.

3.3.4.2) Versionamento automatico tramite Typst

Il processo di gestione delle configurazioni utilizza uno snippet Typst per gestire automaticamente la versione corrente del documento e collegarla ai metadati. Lo snippet è il seguente:

```
#let versionNumber=currentVersion.values().map(n=>{str(n)}).join(".")
#metadata(versionNumber)<versionNumber>
#let doc="Norme di Progetto"
#let versionNumber=currentVersion.values().map(n=>{str(n)}).join(".")
```

- Converte le componenti numeriche della versione (X, Y, Z) in stringhe e le unisce nel formato X.Y.Z. Il risultato viene salvato nella variabile versionNumber, che rappresenta la versione corrente del documento.
- Inserisce la versione corrente nei metadati del documento.
- Permette di rendere il numero di versione accessibile in tutto il documento, per esempio in header, footer o tabella del registro modifiche, senza doverlo aggiornare manualmente.
- Definisce il nome del documento come variabile, utile per riferimenti automatici all'interno del registro modifiche o per generare link e riferimenti interni.

3.3.4.3) Registro modifiche

Il registro modifiche è la tabella principale presente in ogni documento che tiene traccia della versione, degli autori, dei verificatori e delle modifiche effettuate. Ogni voce del registro include:

1. **Vers.** – Versione corrente del documento (X.Y.Z);
2. **Data** – Data di aggiornamento della versione;
3. **Autore** – Persona che ha redatto o modificato il documento;
4. **Verificatore** – Persona che ha revisionato o approvato la modifica;
5. **Descrizione** – Sintesi delle modifiche apportate.

Vers.	Data	Autore	Verificatore	Descrizione
0.9.0	2026-01-10	Nome di chi ha modificato	Nome di chi ha verificato	Stesura Sezione 4.2, Gestione dell'infrastruttura

3.3.4.4) Formato nome dei verbali

Al fine di avere ordine cronologico all'interno della cartella dei verbali, è stato deciso di adottare il seguente standard per la nomina dei verbali. Di questi documenti interessa data e versione, dunque saranno nel formato:

YYYY-MM-DD_Verbale-vX.Y.Z.typ

3.3.5) Strumenti principali utilizzati

Per la gestione delle configurazioni, delle versioni e della documentazione il gruppo utilizza i seguenti strumenti:

- **Typst**: compilazione dei documenti, template riutilizzabili, gestione dei metadati e preview istantanea.
- **GitHub**: repository centralizzato, controllo versioni, gestione issue e collaborazione asincrona.
- **GitHub Actions**: compilazione automatica dei documenti in PDF e aggiornamento della versione corrente.
- **Registro delle Modifiche** (integrato nei documenti tramite Typst): tiene traccia di autori, verificatori, date e descrizione di ogni modifica.

3.4) Processo di Accertamento della Qualità

3.4.1) Introduzione

Il processo di Accertamento Qualità ha il compito di verificare che documenti, deliverable e processi siano conformi agli standard, alle procedure e alle metriche definite dal progetto. Tale processo accompagna l'intero ciclo di vita del progetto e fornisce un controllo continuo sulla qualità del lavoro svolto.

3.4.2) Scopo del processo

Assicurare che il lavoro del team sia:

- **Corretto e coerente** con le specifiche definite;
- **Conforme agli standard e alle metriche** di qualità stabilite;
- **Tracciabile e verificabile**;
- Adeguatamente **preparato per le successive attività** di Qualifica.

3.4.3) Attività del processo

- Revisione della documentazione e dei deliverable;
- Verifica del rispetto delle Norme di Progetto e degli standard adottati;
- Individuazione e segnalazione di eventuali non conformità;
- Raccolta e archiviazione delle evidenze di controllo.

3.4.4) Procedure operative

3.4.4.1) Scrittura dei commit

I commit devono avere un **tipo** ed una **descrizione**:

- Il **tipo** indica qual è l'obiettivo del commit (ad esempio feat, fix, docs, ecc.);
- La **descrizione** aiuta il lettore a comprendere meglio quali cambiamenti sono stati effettuati.

Le regole generali sono:

- Iniziare il commit con il tipo seguito da :
- Lasciare uno spazio tra tipo e descrizione
- Iniziare la descrizione con lettera maiuscola
- Limitare la descrizione a massimo 50 caratteri
- Indicare alla fine del commit la issue a cui ci si sta riferendo

```
git commit -m "tipo: descrizione. Issue #1 "
```

Nel caso sia necessario modificare un commit (ad esempio in caso di errori) si utilizza il seguente comando³:

```
git commit --amend
```

³È consigliato l'utilizzo del comando per modificare commit in locale prima di fare push nella repository condivisa. È preferibile astenersi dal modificare commit che sono già stati resi pubblici.

3.4.5) Strumenti

- **Typst** → Calcolo automatico di alcune metriche principali (es. indice Gulpease) tramite script Typst;
- **GitHub** → repository condiviso, issue tracking e versionamento delle modifiche.
- **Automazioni** → script e workflow per calcolare metriche di qualità, generare report automatici, notificare approvazioni.

3.4.6) Metriche

Le metriche relative all'Accertamento Qualità sono riportate nella sezione **Sezione 5.1** "Metriche di Qualità" del documento, con valori accettabili e ottimali.

Oppure consultare il seguente documento per approfondimenti : [Piano di Qualifica \(PdQ\)](#).

3.5) Processo di Qualifica

3.5.1) Introduzione

Il processo di Qualifica coincide con l'insieme delle attività di Verifica e Validazione pianificate nel progetto. Il suo scopo è dimostrare, tramite evidenze oggettive e misurabili, che il prodotto sia conforme alle specifiche software (Verifica) e alle attese dell'utente (Validazione). In termini operativi, la Qualifica non è solo una fase finale, ma un processo continuo regolato dal Piano di Qualifica, che definisce gli obiettivi quantitativi, e monitorato attraverso un cruscotto di valutazione.

3.5.2) Scopo del processo

- Assicurare la **conformità dei prodotti** alle specifiche tecniche e funzionali.
- **Ridurre errori e incoerenze** attraverso attività di verifica e validazione.
- Documentare in modo chiaro l'esito delle **attività di verifica e validazione**.
- Definire criteri oggettivi di completamento tramite la **Definition of Done (DoD)**.

3.5.3) Attività del processo

Le attività principali del processo di Qualifica comprendono:

- **Revisione finale dei deliverable** rispetto ai requisiti e agli standard;
- **Validazione** dei documenti e dei prodotti software;
- **Verifica del soddisfacimento** dei criteri di completamento;
- **Approvazione finale** dei deliverable qualificati.

3.5.3.1) Analisi statica

L'analisi statica è una tecnica di verifica applicata senza eseguire il codice o il prodotto. Il suo scopo è individuare problemi di sintassi, logica o conformità agli standard prima che si manifestino durante l'esecuzione.

Il gruppo adotta due metodi di lettura a seconda del contesto:

- **Walkthrough:** revisione libera e approfondita dell'intero artefatto, condotta senza una lista di controllo predefinita. Parte dall'ipotesi che esista un difetto, ma senza conoscerne la natura o la posizione. È indicata per revisioni critiche o per artefatti nuovi, dove non è ancora disponibile esperienza pregressa sugli errori tipici. A causa dell'elevato costo in termini di tempo e risorse, viene applicata selettivamente, ad esempio in occasione delle revisioni RTB e PB o per la prima stesura di documenti strutturali.
- **Ispezione:** verifica guidata da una checklist predefinita, focalizzata su errori ricorrenti e criteri di qualità specifici. È meno approfondita del walkthrough, ma facilmente automatizzabile e ripetibile. Viene adottata come metodo principale per la verifica sistematica di documenti e codice, in particolare per verbali, analisi dei requisiti e sezioni già consolidate. Un esempio di checklist adottata è la **Sezione 3.5.3.5** (DOD).

3.5.3.1.1) Checklist di ispezione

La checklist utilizzata dal gruppo copre i seguenti aspetti:

- **Documenti:** ortografia e grammatica, rispetto del template, presenza e correttezza del registro delle modifiche, coerenza dei riferimenti interni, nomenclatura conforme alle Norme di Progetto.
- **Codice:** rispetto delle convenzioni di nomenclatura, assenza di codice morto o commentato, presenza di commenti significativi, gestione degli errori, conformità alle linee guida del linguaggio adottato.

3.5.3.1.2) Quando usare quale metodo

Contesto	Metodo	Motivazione
Prima stesura di un documento critico	Walkthrough	Nessuna checklist disponibile, alto rischio di errori strutturali
Verifica verbali e documenti ricorrenti	Ispezione	Formato standardizzato, errori prevedibili
Revisione del codice in pull request	Ispezione	Automatizzabile, ripetibile, integrata nel workflow Git
Revisione pre-RTB / pre-PB	Walkthrough	Qualità critica, revisione completa necessaria

Tabella 1: Metodologie di testing

3.5.3.2) Analisi dinamica

L'analisi dinamica verifica il software eseguendolo con test specifici, al fine di misurarne la qualità funzionale e individuare errori nel comportamento a runtime. Ogni test è definito da uno stato iniziale, un insieme di input e gli output attesi, e deve produrre risultati riproducibili indipendentemente da chi lo esegue o quando.

I test devono essere ripetibili e automatizzabili, così da poter essere eseguiti in modo continuo durante l'intero ciclo di vita del prodotto, in particolare ad ogni modifica rilevante del codice.

3.5.3.3) Classificazione dei test

Tutti i test pianificati, eseguiti e i loro esiti sono tracciati nel Piano di Qualifica. I test sono classificati in categorie gerarchiche, ciascuna identificata da un codice univoco nel formato:

TipoTest_XX

dove **TipoTest** indica la categoria e **XX** è un numero progressivo che identifica univocamente il test all'interno di essa.

Le categorie previste sono:

Codice	Descrizione
TU	Test di Unità — verifica il corretto funzionamento di una singola unità software in isolamento Divisi a loro volta in: • TU-F: per i test del frontend. • TU-B: per i test del backend.
TI	Test di Integrazione — verifica l'interazione corretta tra più unità o componenti Divisi a loro volta in: • TI-F: per i test del frontend. • TI-B: per i test del backend.
TS	Test di Sistema — verifica il comportamento del sistema nella sua interezza rispetto ai requisiti
TA	Test di Accettazione — verifica che il prodotto soddisfi i criteri concordati con il committente

Tabella 2: Tipi di test

3.5.3.3.1) Stati dei test

Per consentire un monitoraggio efficace dell'avanzamento, ogni test assume uno dei seguenti stati:

Stato	Descrizione
P	Passed — il test è stato eseguito e ha prodotto l'output atteso
I	Implementato — il test è stato scritto ma non ancora eseguito
NI	Non Implementato — il test è pianificato ma non ancora realizzato
F	Fallito — il test ha prodotto un risultato diverso da quello atteso

Tabella 3: Stati dei test

3.5.3.3.2) Test di Unità

I test di unità rappresentano il livello di granularità più basso e verificano il corretto funzionamento delle singole unità software nella loro forma più elementare: funzioni, metodi o classi.

Si distinguono due approcci:

- **Test funzionali**: analizzano il comportamento dell'unità osservando esclusivamente input e output, senza considerare la logica interna di implementazione. Poiché non esaminano i percorsi di esecuzione interni, non sono sufficienti da soli a garantire la correttezza dell'unità e devono essere affiancati da test strutturali.

- **Test strutturali:** analizzano la logica interna del codice, verificandone i diversi percorsi di esecuzione e la relativa copertura. Consentono di individuare rami non raggiungibili, condizioni mal gestite o percorsi di errore non testati.

I due approcci sono complementari: i test funzionali verificano il contratto esterno dell'unità, quelli strutturali ne garantiscono la solidità interna.

3.5.3.3.3) Test di Integrazione

I test di integrazione verificano che le componenti del sistema, già testate singolarmente, interagiscano e comunichino correttamente tra loro. L'obiettivo è individuare errori che emergono solo dalla combinazione di moduli: incompatibilità di interfacce, gestione scorretta dei dati condivisi o comportamenti inattesi nelle sequenze di chiamata.

Il gruppo adotta uno dei due approcci seguenti a seconda della struttura del componente da integrare:

Approccio	Descrizione	Quando usarlo
Top-Down	Si parte dalle componenti di livello superiore, utilizzando stub per simulare quelle non ancora integrate	Quando la logica di controllo ad alto livello è prioritaria o le componenti di base non sono ancora disponibili
Bottom-Up	Si parte dalle componenti di base, utilizzando driver per simulare le chiamate dei livelli superiori	Quando le componenti fondamentali sono stabili e si vuole verificarne il comportamento prima di procedere verso l'alto

Tabella 4: Tipi di test d integrazione

3.5.3.3.4) Test di Sistema

I Test di Sistema verificano il comportamento del sistema nella sua interezza, valutandone la conformità rispetto ai requisiti funzionali e non funzionali definiti nel documento di Analisi dei Requisiti. Consentono di validare aspetti quali la correttezza funzionale, l'affidabilità, la robustezza e la gestione degli errori in condizioni reali o simulate di utilizzo.

3.5.3.3.5) Test di Accettazione

I Test di Accettazione verificano che il sistema soddisfi le aspettative e i requisiti richiesti dalla proponente BlueWind Srl. Vengono eseguiti al termine del ciclo di sviluppo e coinvolgono direttamente la proponente nella valutazione del prodotto consegnato. Il loro esito positivo costituisce condizione necessaria per la validazione finale.

3.5.3.3.6) Test di Regressione

I Test di Regressione verificano che le modifiche apportate al sistema — siano esse correzioni di difetti o aggiunta di nuove funzionalità — non abbiano compromesso comportamenti precedentemente testati e corretti.

In tal caso il processo da seguire è:

1. Analizzare il problema e identificarne la causa;
2. Sviluppare e codificare la soluzione;
3. Rieseguire il test fallito per verificare che il problema sia stato risolto;
4. Rieseguire **l'intera suite di test** per assicurarsi che la correzione non abbia introdotto regressioni.

Quest'ultimo punto è fondamentale: una modifica apparentemente localizzata può avere effetti inattesi su funzionalità già verificate. Per questo motivo non ci si limita al solo test fallito, ma si rieseguono tutti i test disponibili.

3.5.3.4) Validazione

Il processo di Validazione ha lo scopo di accertare che quanto realizzato soddisfi le esigenze di BlueWind Srl. Si distingue dalla verifica per il suo focus sul risultato finale: mentre la verifica accerta che il prodotto sia costruito correttamente («*Are we building the system right?*»), la validazione accerta che sia stato costruito il prodotto corretto («*Are we building the right system?*»). Dunque, pur avendo responsabilità diverse, la verifica prepara il successo della validazione. Sebbene la verifica si applichi ai singoli prodotti intermedi e la validazione al sistema nel suo complesso, la prima fornisce il supporto e l'evidenza oggettiva necessari per sostenere la successiva conclusione che il software sia validato e pienamente conforme alle attese del committente.

3.5.3.4.1) Attività di validazione

La Validazione si basa sull'analisi degli esiti dei test di accettazione e sul tracciamento dei requisiti definiti in accordo con BlueWind Srl. L'obiettivo è confermare che ogni requisito obbligatorio sia implementato e che il sistema si comporti correttamente in relazione a ciascuno di essi.

3.5.3.5) Definition of Done

La **Definition of Done (DoD)** è un elemento molto importante nello sviluppo software, perché definisce le azioni che devono essere completate affinché i requisiti — espressi tramite un **Product Backlog Item (PBI)** — siano considerati conclusi.

I criteri che la compongono devono essere concreti, verificabili e di dimensione ridotta, e hanno l'obiettivo di garantire un livello minimo di qualità per ogni rilascio o incremento del prodotto.

La seguente **Definition of Done** non sono statiche, ma dinamiche: evolvono in base alle esigenze del team di sviluppo.

3.5.3.5.1) Definition of Done - Ambito documentale

Di seguito viene riportata la Definition of Done per ogni prodotto documentale:

- ☐ Controllare a livello semantico e grammaticale che tutto sia corretto (grammatica, punteggiatura, sintassi, rivedere frasi ripetute/ mal espresse);
- ☐ Controllare di aver incluso tutte le sezioni definite del WoW nel documento su cui si lavora;
- ☐ Controllare di aver aggiornato nella tabella riepilogativa dello stato del documento:
 - stato;
 - versione;
 - ruoli.
- ☐ Controllare di aver aggiunto le ultime modifiche anche sulla “tabella delle modifiche del documento”;
- ☐ **Nei verbali**, controllare che tutte le decisioni corrispondano a issue specifiche nell’issue template;
- ☐ L’avvenuto completamento della attività documentale deve essere sancito tramite un commit di chiusura che referenzi la issue corrispondente con

```
git commit -m "commento. Close #numero_issue"
```

Verificare poi effettivamente la chiusura nel Projects Board.

Una volta verificati tutti i criteri precedenti, il Responsabile approva il lavoro svolto spostando le relative issue nello stato di “Done”. Solo quando tutti i PBI previsti per lo Sprint risultano completati, il Responsabile autorizza il merge del branch develop nel branch main.

3.5.3.5.2) Definition of Done - Ambito codice

Di seguito viene riportata la Definition of Done per le attività di sviluppo del codice sorgente:

- ☐ Controllare che l’implementazione rispetti i requisiti definiti e l’architettura stabilita nella fase di progettazione iniziale;
- ☐ Verificare che, in caso di modifiche architetturali o logiche avvenute durante la codifica, la relativa documentazione di progetto sia stata aggiornata di conseguenza;
- ☐ Controllare che il codice rispetti le norme di codifica e le convenzioni di stile adottate dal team;
- ☐ Controllare che i test (unitari e/o di integrazione) siano stati scritti, eseguiti e superati con successo per le funzionalità implementate, in caso contrario il verificatore dovrà integrarli;

- ☐ Verificare la correttezza del codice tramite gli appositi tool e verificare il superamento dei test relativi alla continuous integration;
- ☐ Controllare che il codice sia stato sottoposto a Code Review tramite l'apertura di una Pull Request (PR) e approvato da almeno un altro membro del team;
- ☐ L'avvenuto completamento della attività di sviluppo deve essere sancito tramite un commit di chiusura che referenzi la issue corrispondente con

```
git commit -m "commento. Close #numero_issue"
```

Verificare poi effettivamente la chiusura nel Projects Board.

Al termine dello sprint il responsabile autorizza la pubblicazione delle funzionalità che soddisfano i criteri precedentemente definiti e procede alla pubblicazione sul branch main.

3.5.4) Strumenti a supporto

- **Typst** → gestione dei documenti, versionamento e inserimento automatico di metadati.
- **GitHub** → repository condiviso, issue tracking e tracciamento delle modifiche.
- **Automazioni** → script e workflow per calcolare metriche di qualità, generare report automatici, notificare approvazioni.

3.5.5) Documentazione a supporto

I processi di verifica e validazione si appoggiano ai seguenti documenti:

- **Analisi dei Requisiti**: definisce i requisiti funzionali e non funzionali concordati con BlueWind Srl. Costituisce la base di riferimento per la progettazione dei test di sistema e di accettazione, e per il tracciamento della copertura dei requisiti.

[Riferimento all'Analisi dei Requisiti.](#)

- **Piano di Qualifica**: raccoglie le metriche di qualità adottate, i test pianificati ed eseguiti e i loro esiti. È il documento operativo di riferimento per il monitoraggio dell'avanzamento delle attività di verifica.

[Riferimento al Piano di Qualifica](#)

- **Piano di Progetto**: definisce la pianificazione temporale delle attività, incluse quelle di verifica. Consente di contestualizzare i risultati dei test rispetto agli sprint in cui sono stati eseguiti.

[Riferimento al Piano di Progetto.](#)

- **Norme di Progetto:** definisce le modalità operative di verifica e validazione adottate dal gruppo, incluse le checklist di ispezione, la classificazione dei test e i criteri di accettazione.

[Riferimento alle Norme di Progetto.](#)

- **Verbali esterni:** documentano le decisioni e i requisiti concordati con BlueWind Srl nel corso delle riunioni. Costituiscono riferimento per verificare la corrispondenza tra quanto richiesto e quanto implementato.

[Riferimento ai Verbali esterni](#)

4) Processi Organizzativi

4.1) Introduzione

I processi organizzativi definiscono il quadro operativo entro cui il progetto viene pianificato, gestito e migliorato nel tempo. Essi supportano i processi primari e di supporto garantendo **coerenza, continuità operativa e qualità complessiva**, in accordo con gli standard di riferimento per il ciclo di vita del software.

Nel presente progetto sono stati individuati i seguenti processi organizzativi:

1. Processo di gestione del processo

Stabilisce e governa le modalità operative del progetto, definendo ruoli, responsabilità, procedure e regole condivise per l'esecuzione delle attività;

2. Processo di gestione dell'infrastruttura

Assicura la disponibilità, l'adeguatezza e il corretto utilizzo degli strumenti software e delle risorse tecnologiche necessarie allo svolgimento del progetto;

3. Processo di miglioramento

Volto a valutare periodicamente l'efficacia dei processi adottati e ad apportare azioni correttive e migliorative sulla base di metriche, evidenze e risultati ottenuti;

4. Processo di formazione

Finalizzato a mantenere e accrescere le competenze del team, garantendo che ogni membro disponga delle conoscenze necessarie per svolgere i compiti assegnati.

L'adozione strutturata di tali processi consente al gruppo di operare in modo ordinato, ripetibile e orientato al miglioramento continuo, riducendo i rischi organizzativi e aumentando l'affidabilità del progetto.

4.2) Processo di gestione del processo

4.2.1) Introduzione

Il Processo di Gestione del Processo definisce il quadro organizzativo entro cui vengono pianificate, coordinate e monitorate tutte le attività di progetto.

Esso fornisce le linee guida per governare l'evoluzione del progetto, assicurando coerenza tra i processi adottati, le risorse disponibili e gli obiettivi stabiliti, e supporta l'interazione tra i diversi processi organizzativi, primari e di supporto.

4.2.2) Scopo del processo

Lo scopo del processo di Gestione del Processo è:

- **Definire e strutturare le attività** necessarie allo svolgimento del progetto;
- **Assegnare responsabilità e ruoli** in modo chiaro e coerente;
- **Pianificare e monitorare l'avanzamento** delle attività;

- **Garantire una comunicazione efficace** sia interna al gruppo sia verso gli stakeholder esterni;
- Assicurare il controllo del progetto nel **rispetto di tempi, costi e standard di qualità** stabiliti.

4.2.3) Attività del processo

Lo standard [ISO/IEC/IEEE 12207:1995](#) definisce la gestione del processo come un insieme di attività organizzate che permettono di pianificare, controllare e valutare l'esecuzione dei processi di progetto.

Nel contesto del progetto, tali attività coprono l'intero ciclo di vita del processo e sono strutturate in fasi successive, ciascuna con obiettivi e responsabilità ben definite. Esse consentono di garantire il corretto avanzamento delle attività, il rispetto delle risorse disponibili e la conformità agli standard di qualità stabiliti.

Le attività individuate dallo standard e adottate dal gruppo sono:

1. Inizializzazione

Rappresenta la prima fase del processo. In questa fase vengono definiti i requisiti del processo oggetto di analisi. Una volta stabiliti, il responsabile ne valuta la fattibilità in relazione alle risorse disponibili.

2. Pianificazione

Il responsabile pianifica le attività del processo.

I piani definiscono le attività e i task associati, nonché le caratteristiche del prodotto software.

Ogni piano deve includere le seguenti informazioni:

- tabella di marcia per il completamento dei task;
- stima dello sforzo richiesto;
- risorse necessarie;
- assegnazione dei compiti;
- assegnazione delle responsabilità (si veda la sezione **Sezione 4.2.4.1**);
- quantificazione dei rischi;
- metriche di controllo della qualità;
- costi associati all'esecuzione del processo;
- fornitura dell'ambiente e delle infrastrutture.

3. Esecuzione e controllo

Il responsabile avvia le attività del processo in conformità a quanto stabilito nella fase di pianificazione e ne monitora l'andamento.

- **Controllo interno:** verifica il progresso delle attività e ne mantiene la tracciabilità. Eventuali criticità o impedimenti che possano causare ritardi devono essere segnalati tempestivamente al responsabile.

- **Comunicazione esterna:** gestisce i flussi comunicativi con la **proponente** e il **committente**, garantendo chiarezza e coerenza informativa.

4. Revisione e valutazione

Al completamento delle attività, il verificatore controlla che i risultati ottenuti siano conformi alle metriche di qualità e agli standard definiti.

5. Chiusura

Un'attività si ritiene completa dopo aver superato l'attività di verifica. Si veda la Definition of Done (**Sezione 3.5.3.5**) per maggiori dettagli. La chiusura delle issue legate alle attività avviene tramite merge sul main a intervalli prefissati.

4.2.4) Procedure operative

4.2.4.1) Ruoli di Progetto

La seguente sezione descrive la distribuzione delle responsabilità all'interno del team. Per garantire coerenza, efficienza e qualità, ogni ruolo assolve compiti specifici all'interno dei processi previsti dal Way of Working.

Coerentemente con gli obiettivi del corso di Ingegneria del Software, la struttura organizzativa permette una rotazione tale per cui ogni membro del team ricopra ciascun ruolo almeno una volta.

La tabella sottostante riassume le responsabilità e l'apporto di ogni figura:

Ruolo	Compiti	Presenza
Responsabile	- Coordinamento piani e scadenze - Approvazione release - Comunicazione col committente - Uso efficiente delle risorse - Approvazione/redazione documenti	Tutto il progetto
Amministratore	- Garanzia efficienza strumenti - Gestione tecnologie di supporto - Verifica procedure secondo norme	Tutto il progetto
Verificatore	- Testing e validazione - Controllo qualità deliverable - Conformità ai requisiti	Tutto il progetto
Analista	- Analisi dei requisiti - Definizione bisogni del sistema - Redazione specifiche funzionali	Massimo nella definizione dei requisiti; continuo per l'affinamento
Progettista	- Progetta architettura sistema - Design e modellazione - Traduzione dei requisiti in struttura tecnica	Progettazione architetture e di dettaglio
Programmatore	- Codifica software - Implementazione design - Sviluppo funzionalità	Implementazione

4.2.4.2) Rendicontazione delle ore

La pianificazione e il monitoraggio delle ore produttive del progetto sono gestiti tramite il documento «Piano di Progetto», in cui vengono registrate sia le ore previste sia quelle effettivamente svolte per ciascun ruolo e membro del gruppo.

La ripartizione oraria è accompagnata dai relativi costi, consentendo una visione completa delle risorse economiche impiegate. Al fine di rendere rigorosa, oggettiva e verificabile la rendicontazione oraria, il team ha implementato un foglio di calcolo Google Sheets avanzato dedicato al tracciamento dell'effort. Per alimentare questo documento, è stato sviluppato uno script automatizzato che estrae i dati dal repository e genera dei file CSV dedicati per ogni singolo sprint, i quali vengono poi importati nel foglio. Questo strumento aggrega i dati operativi con lo storico e la gestione delle issue (dimensione, tipologia e scadenze). Tale sistema integrato permette non solo di mappare con precisione l'impegno effettivo di ciascun membro, ma anche di classificare le ore in produttive e non produttive, scalandole dinamicamente dal budget preventivato per i vari ruoli di progetto. Questa organizzazione strutturata favorisce una gestione chiara, trasparente e collaborativa della distribuzione dei ruoli all'interno del team.

4.2.4.3) Assegnazione Ruolo-Documento

La seguente sezione chiarisce i documenti associati a ciascun ruolo.

L'assegnazione viene rappresentata tramite una **legenda** e una **tabella riassuntiva**.

Legenda :

Azioni:

- R = Redazione.
- V = Verifica.
- A = Approvazione.
- C = Contribuente.

Ruoli:

- Responsabile = RESP.
- Amministratore = AMM.
- Analista = ANL.
- Progettista = PRG.
- Programmatore = PGRmm.
- Verificatore = VRF.

Documento	RESP	AMM	ANL	PRG	PGRmm	VRF
Norme di Progetto (NdP)	R/A	R	-	-	-	V
Analisi dei Requisiti (AdR)	A	-	R	-	-	V

Documento	RESP	AMM	ANL	PRG	PGRmm	VRF
Piano di Progetto (PdP)	R/A	-	C (supporto rischi)	-	-	V
Piano di Qualifica (PdQ)	A	C (configurazione dei tool automatici)	-	-	-	R/V (Verificatore A redige, il Verificatore B verifica)
Specifica Tecnica	A	-	C (per coerenza requisiti)	R	C	V
Manuale Utente (MU)	A	-	-	R	C	V
Verbali interni	R/A	-	-	-	-	V
Verbali esterni	R/A	-	-	-	-	V

4.2.4.4) Issue Tracking System

L'Issue Tracking System è lo strumento utilizzato dal team per pianificare, assegnare, monitorare e controllare l'avanzamento delle attività di progetto. Il sistema è accessibile tramite la repository GitHub e si basa su un template di issue condiviso, così da garantire uniformità, tracciabilità e chiarezza operativa.

Ogni issue rappresenta un'attività, un task o una modifica da realizzare e segue un flusso di stato predefinito:

- **Backlog:** attività pianificate ma non ancora avviate;
- **In lavorazione:** attività attualmente in esecuzione;
- **In verifica:** attività completate e sottoposte a controllo;
- **In approvazione:** attività in attesa di validazione finale;
- **Done:** attività concluse e integrate.

Le issue possono essere organizzate in modo gerarchico tramite **relazioni parent/child**, consentendo di suddividere attività complesse in sotto-attività più semplici e gestibili.

La chiusura di un'issue avviene esclusivamente al soddisfacimento dei criteri definiti nella Definition of Done (**Sezione 3.5.3.5**), che garantisce il rispetto degli standard di qualità e di completezza stabiliti.

Inoltre, il sistema di gestione delle issue prevede:

- La **definizione delle milestone**, utilizzate per raggruppare attività e monitorare il raggiungimento degli obiettivi intermedi;

- L'**uso della retrospettiva**, al termine di iterazioni o sprint, per analizzare le attività completate, individuare criticità e definire azioni di miglioramento per le fasi successive.

Questo approccio consente una gestione strutturata, trasparente e verificabile dell'intero ciclo di vita delle attività di progetto.

4.2.4.4.1) Creazione e struttura di un'issue

4.2.4.4.1.1) Creazione e struttura di un'issue - Documentazione

La creazione delle issue avviene a seguito di riunioni interne o incontri con la proponente, durante i quali il gruppo individua le attività su cui concentrarsi. L'**amministratore** è responsabile della creazione delle issue nel sistema, utilizzando l'apposito **template** definito dal gruppo.

Ogni nuova issue deve includere le seguenti informazioni:

1. Assegnatario/i

Generalmente l'issue viene assegnata a una singola persona. In casi particolari, come attività di formazione o di esercitazione (**palestra**), l'issue può essere assegnata a più membri o all'intero gruppo.

2. Descrizione

Una descrizione chiara, dettagliata e non ambigua delle attività da svolgere.

3. Scopo

Specifica il risultato atteso al termine dell'issue e indica dove tale risultato deve essere documentato (ad esempio un documento, una sezione specifica o una modifica al repository).

4. Autore

Il membro del gruppo incaricato di svolgere l'issue.

5. Verificatore

Il membro incaricato di verificare il corretto completamento dell'issue secondo i criteri definiti nella **Definition of Done (Sezione 3.5.3.5)**. Per garantire l'oggettività della verifica e l'assenza di conflitti di interesse, il verificatore deve essere necessariamente una figura diversa dall'autore.

6. Label (ambito/destinazione)⁴

Le label permettono di classificare le issue in base al loro ambito all'interno del progetto, facilitandone l'organizzazione e la ricerca. Le principali label adottate sono:

- Analisi dei Requisiti,

⁴Le label possono essere aggiornate nel corso del progetto: label non più utili possono essere rimosse e nuove label introdotte in base alle esigenze.

- Piano di Progetto,
- Piano di Qualifica,
- Norme di Progetto,
- Verbale,
- Diario di Bordo,
- Glossario,
- Generale → attività non direttamente riconducibili ai documenti principali (studio di materiale, gestione repository, sito web, attività varie).

7. Tipo di issue (Type)

Consente di distinguere la natura dell'attività:

- **Palestra** → attività formative non rendicontate;
- **Produttivo** → attività rendicontate che producono risultati concreti;
- **Bug** → individuazione e risoluzione di errori o malfunzionamenti;
- **Correzione** → modifiche e miglioramenti a materiali o documenti esistenti.

8. Priorità (Bassa, Media, Alta)

La priorità ha un duplice scopo:

- supportare la valutazione dell'importanza dell'issue;
- comunicare all'assegnatario il livello di urgenza dell'attività.

9. Dimensione (ExtraSmall, Small, Medium, Large)

Fornisce una stima indicativa della quantità di lavoro necessaria per completare l'issue.

10. Data di scadenza

Generalmente coincide con la fine dello sprint di riferimento o con una milestone pianificata.

4.2.4.4.1.2) Creazione e struttura di un'issue - Codice

La creazione delle issue avviene a seguito di riunioni interne o incontri con la proponente, durante i quali il gruppo individua nuove attività. L'**amministratore** è responsabile della creazione delle issue nel sistema, utilizzando l'apposito **template** definito dal gruppo.

A seguito delle attività di progettazione sono state create un cospicuo numero di issue, una per ogni classe progettata. Queste sono state inserite nel backlog e solo quelle pronte sono state inserite in ready, lo spostamento avviene periodicamente.

Ogni nuova issue deve includere le seguenti informazioni:

1. Assegnatario/i

Generalmente l'issue viene assegnata a una singola persona. In casi particolari, come attività di formazione o di esercitazione (**palestra**), l'issue può essere assegnata a più membri o all'intero gruppo.

2. Descrizione

Una descrizione chiara, dettagliata e non ambigua delle attività da svolgere.

3. Autore

Il membro del gruppo incaricato di svolgere l'issue.

Deve allegare al codice creato anche dei test.

4. Verificatore

Il membro incaricato di verificare il corretto completamento dell'issue secondo i criteri definiti nella **Definition of Done (Sezione 3.5.3.5)**. Per garantire l'oggettività della verifica e l'assenza di conflitti di interesse, il verificatore deve essere necessariamente una figura diversa dall'autore.

Deve aggiungere test mancanti se lo ritiene necessario.

5. Label (ambito/destinazione)⁵

Le label permettono di classificare le issue in base al loro ambito all'interno del progetto, facilitandone l'organizzazione e la ricerca. Le principali label adottate sono le seguenti:

Bug:

Indica un bug da correggere

Documentation:

Segnala la necessità dell'aggiornamento della documentazione

Frontend:

Indica un issue relativo all'implementazione del sistema frontend

Inbound adapter/controller:

Indica un issue relativo all'implementazione di un controller del sistema backend

Inbound port:

Indica un issue relativo alla definizione di una porta inbound.

Service:

Indica un issue relativo all'implementazione di un service

Outbound port:

Indica un issue relativo alla definizione di una outbound port

Outbound adapter:

Indica un issue relativo all'implementazione di un outbound adapter

Vue component:

Indica un issue relativo all'implementazione di un componente Vue

Vue store:

Indica un issue relativo all'implementazione di una parte dello store Vue

⁵Le label possono essere aggiornate nel corso del progetto: label non più utili possono essere rimosse e nuove label introdotte in base alle esigenze.

6. Tipo di issue (Type)

Consente di distinguere la natura dell'attività:

- **Palestra** → attività formative non rendicontate;
- **Produttivo** → attività rendicontate che producono risultati concreti;
- **Bug** → individuazione e risoluzione di errori o malfunzionamenti;
- **Correzione** → modifiche e miglioramenti a materiali o documenti esistenti.

7. Priorità (Bassa, Media, Alta)

La priorità ha un duplice scopo:

- supportare la valutazione dell'importanza dell'issue;
- comunicare all'assegnatario il livello di urgenza dell'attività.

8. Dimensione (ExtraSmall, Small, Medium, Large)

Fornisce una stima indicativa della quantità di lavoro necessaria per completare l'issue.

9. Data di scadenza

Generalmente coincide con la fine dello sprint di riferimento o con una milestone pianificata.

4.2.4.4.2) Flusso operativo

Il flusso operativo standard per la gestione di un'issue è il seguente:

1. Il responsabile individua le attività necessarie al raggiungimento degli obiettivi; l'amministratore le traduce operativamente creando le issue sul repository tramite i template predisposti;
2. Vengono assegnati autore/i e verificatore/i;
3. Vengono compilati tutti i campi richiesti (descrizione, scopo, label, tipo, priorità, dimensione, scadenza);
4. L'issue viene inserita nello stato iniziale Backlog;
5. L'issue avanza attraverso gli stati del workflow fino al completamento;
6. La chiusura definitiva dell'issue avviene tramite un commit di chiusura e lo spostamento manuale in Done nel Project Board, operazione che spetta al responsabile come atto di approvazione finale dell'attività.

4.3) Processo di Gestione dell'infrastruttura

4.3.1) Introduzione

Questo processo definisce, realizza e mantiene l'insieme delle infrastrutture e degli strumenti di supporto utilizzati dal team durante il ciclo di vita del progetto.

4.3.2) Scopo del processo

Esso ha lo scopo di garantire un ambiente di lavoro affidabile, coerente e condiviso, che consenta lo svolgimento efficace delle attività di pianificazione, sviluppo, verifica e gestione. L'adozione e la manutenzione controllata degli strumenti assicurano dispo-

nibilità, aggiornamento continuo e conformità agli standard di progetto, riducendo il rischio di inefficienze operative e problemi organizzativi.

4.3.3) Attività del processo

Questa sezione descrive le macro-attività del processo di infrastruttura, indipendentemente dagli strumenti specifici.

Il processo di infrastruttura comprende le seguenti attività principali:

1. Implementazione del processo

Definizione e adozione degli strumenti necessari a supportare le attività del progetto, con particolare attenzione alla comunicazione, alla collaborazione e all'automazione.

2. Creazione

Configurazione iniziale degli strumenti selezionati, predisposizione degli ambienti di lavoro e definizione delle strutture organizzative (repository, template, workflow).

3. Manutenzione

Aggiornamento e gestione continua dell'infrastruttura al fine di garantirne affidabilità, coerenza e conformità agli standard di progetto.

Tutti i task relativi alle attività di creazione e manutenzione sono di competenza dell'**Amministratore**.

4.3.4) Procedure operative

4.3.4.1) Implementazione del processo

Per facilitare il lavoro del gruppo, in particolare la comunicazione asincrona e la collaborazione distribuita, sono stati adottati i seguenti strumenti:

Discord – Piattaforma di messaggistica e videoconferenza per riunioni interne da remoto.

Zoom – Piattaforma di videoconferenza per riunioni con la proponente BlueWind.

WhatsApp – Messaggistica istantanea per comunicazioni interne asincrone.

Telegram – Messaggistica asincrona per comunicazioni con la proponente.

Typst – Linguaggio di markup per la redazione dei documenti, con compilazione automatizzata tramite script.

Tinymist Typst – Estensione di Visual Studio Code per il supporto sintattico e la live preview.

Script bash – Automazioni eseguite tramite GitHub Actions.

Git – Sistema di versionamento distribuito per codice e documenti.

GitHub – Piattaforma cloud per repository, issue tracking, integrazione continua e hosting.

Google Docs – Editor collaborativo per attività di brainstorming asincrono.

Google Sheets – Fogli di calcolo per il tracciamento di dati e metriche.

Google Drive – Sistema di file sharing per materiali di progetto.

Python – Linguaggio di programmazione usato per gestire alcuni script di automazione.

4.3.4.2) Creazione

In questa sezione viene descritto il processo di creazione e configurazione degli strumenti ritenuti significativi per il supporto alle attività di progetto.

Typst

L'ambiente di lavoro **Typst** è stato personalizzato per supportare in modo efficiente la redazione della documentazione. Typst consente la definizione di **template**, utilizzati in modo analogo alle chiamate di funzione di un linguaggio di programmazione.

I template principali svolgono le seguenti funzioni:

Impaginazione:

Impostazione uniforme di header e footer e definizione delle regole di stile comuni a tutti i documenti⁶.

Tabelle:

Predisposizione di template per la creazione semplificata di tabelle.

Sprint:

Automazione del riepilogo degli sprint, comprensiva dei calcoli necessari e del layout dedicato.

Marcatura automatica dei termini del Glossario:

Funzionalità attivabile tramite un apposito flag booleano; se lasciata sempre attiva può impattare negativamente le prestazioni della live preview.

Separazione tra contenuto e layout del Glossario:

I termini e le definizioni del Glossario sono mantenuti in un file separato sotto forma di dizionario, consentendo

⁶Ad esempio, i link non sono visualizzati in blu e sottolineati di default; tale regola stilistica viene ereditata automaticamente dal contenuto del documento.

l'ordinamento automatico e la generazione di viste personalizzate in formato PDF e HTML.

Gestione automatica della numerazione :

È stata implementata una gestione automatica della numerazione sia in fase di creazione sia in fase di referenziamento, anche tra documenti diversi. La referenziazione avviene tramite funzioni apposite che accettano in input una stringa rappresentante il nome che identifica l'elemento target.

Gli elementi finora sottoposti a tale numerazione automatica sono:

- Casi d'uso;
- Requisiti;
- Test.

Tracciamento automatico:

Sono state predisposte apposite funzioni per la costruzione automatica delle seguenti tabelle di tracciamento:

- Tracciamento casi d'uso - requisiti funzionali;
- Tracciamento test di sistema - requisiti funzionali.

Plot diagrammi dei casi d'uso:

È stata predisposta una funzione per il plot dei diagrammi di attività in modo da poter applicare l'automazione già predisposta per la gestione del numbering.

Git

L'intero progetto utilizza **Git** come sistema di versionamento.

Sono stati definiti due branch principali:

- **Main**, destinato ai rilasci ufficiali;
- **Develop**, utilizzato per lo svolgimento delle attività di progetto.

È stato inoltre predisposto un file `.gitignore` per evitare la pubblicazione di file indesiderati.

GitHub

È stata creata una [GitHub Organization \(https://github.com/GroupRubberDuck\)](https://github.com/GroupRubberDuck) dedicata alle attività di progetto e i seguenti repository:

- [Documentazione https://github.com/GroupRubberDuck/Documentazione](https://github.com/GroupRubberDuck/Documentazione)
- [Proof of Concept https://github.com/GroupRubberDuck/PoC](https://github.com/GroupRubberDuck/PoC)

	GitHub Actions: Sono state configurate delle GitHub Actions per la compilazione automatica dei file Typst e per l'aggiornamento automatico del sito web. Utilizzate anche per realizzare la continuous integration del repository del codice. GitHub Pages: È stata attivata la funzionalità GitHub Pages per l'hosting del sito web del progetto. GitHub Issue Tracking System: Il gruppo ha deciso di avvalersi dell'issue tracking system offerto da GitHub; maggiori dettagli sono disponibili nella sezione dedicata alla guida operativa Sezione 4.2.4.4.
Strumenti di comunicazione	Nessuno degli strumenti di comunicazione ha richiesto operazioni significative durante la fase di creazione.
Strumenti Google	È stata creata una mail dedicata alle attività di progetto. Google Drive e Google Docs non richiedono particolari operazioni di configurazione, se non il caricamento e la condivisione del materiale. Google Sheets richiede invece operazioni più complesse per l'implementazione delle metriche e degli indicatori stabiliti.
Script python	Insieme alla numerazione automatica realizzata tramite funzioni Typst è stato predisposto uno script python per mantenere ordinati e coerenti con la numerazione i file all'interno della cartella di lavoro. Inoltre crea eventuali file mancanti partendo da un template configurabile e aggiorna il file index usato per l'aggregazione dei singoli elementi. Usato anche per la raccolta delle metriche.
Docker	Al fine di standardizzare l'ambiente di sviluppo si è deciso di configurare un ambiente di sviluppo containerizzato con docker e orchestrato con docker compose.
Docker Compose	
Vite	Tool utilizzato solo in ambito di sviluppo per testare, analizzare e compilare le parti reattive di frontend.
Watcher.sh	Container che usa vite per generare un compile down in javascript puro che permette semplifica l'integrazione sul frontend.

Poetry

Tool usato per la risoluzione e gestione delle dipendenze python.

Tabella 5: Strumenti utilizzati

4.3.4.3) Manutenzione

Nel corso del progetto è necessario garantire il costante aggiornamento e il corretto funzionamento dell'infrastruttura e degli strumenti adottati. Le attività di manutenzione hanno lo scopo di assicurare la continuità operativa, l'affidabilità e l'allineamento agli standard definiti.

Le operazioni di manutenzione possono essere classificate in base alla frequenza:

- **Manutenzione ordinaria:** aggiornamenti periodici e interventi ricorrenti legati alla normale evoluzione delle attività di progetto;
- **Manutenzione non ordinaria:** aggiunte, modifiche o rimozioni di strumenti, configurazioni o risorse rese necessarie da cambiamenti nei requisiti, criticità emerse o decisioni organizzative straordinarie.

4.3.4.3.1) Project board

Le issue vengono organizzate all'interno di una **Project Board** dedicata, che consente di avere una visione d'insieme sullo stato di avanzamento delle attività e di monitorare il carico di lavoro del team.

4.3.4.3.2) Milestone

Le **milestone** rappresentano obiettivi intermedi fondamentali nella pianificazione di medio e lungo periodo.

L'assegnazione delle issue alle milestone è responsabilità dell'amministratore, che provvede inoltre al loro aggiornamento in caso di variazioni nella pianificazione o nelle priorità di progetto.

4.3.4.4) GitHub Actions

Le **GitHub Actions** costituiscono il sistema di automazione adottato dal gruppo per supportare le attività ricorrenti del progetto, in particolare quelle legate alla compilazione e al rilascio della documentazione.

Le azioni sono definite tramite file di configurazione in formato **YAML** e sono memorizzate nella cartella `.github/workflows` del repository. La loro manutenzione e modifica rientra nelle responsabilità dell'amministratore di progetto.

4.3.4.5) Script e automazioni

Il mantenimento degli script e delle automazioni è di competenza dell'**amministratore**.

Tali attività di manutenzione, siano esse ordinarie o straordinarie, si rendono necessarie nei seguenti scenari:

- **Evoluzione:** introduzione di nuove automazioni per supportare processi emergenti;
- **Ottimizzazione:** dismissione di procedure divenute obsolete o ridondanti;
- **Correzione:** risoluzione di malfunzionamenti o errori logici negli script esistenti.

Le automazioni strettamente legate alla redazione dei documenti in **Typst**, inclusi i template, sono salvate nella cartella [src/TypstTemplate](#) all'interno del repository [Documentazione](#).

Gli script di uso generale, non direttamente collegati a Typst, sono invece salvati nella cartella [scripts](#).

4.3.4.6) Discord

Per quanto riguarda la piattaforma **Discord**, le attività operative rilevanti si limitano alla:

- Creazione di nuovi canali, qualora necessario;
- Moderazione di base per garantire ordine e chiarezza nelle comunicazioni interne.

Non sono previste configurazioni avanzate o attività di manutenzione complesse.

4.3.4.7) Strumenti Google

Di seguito sono elencate le principali attività operative associate agli strumenti della suite Google utilizzati nel progetto.

Google Drive:

Gestione dello spazio condiviso, inclusa la creazione di nuove cartelle e l'eliminazione di file obsoleti.

Google Docs:

Predisposizione, ove necessario, di layout coerenti per i documenti condivisi e di supporto alle attività di progetto.

Google Sheets:

Implementazione e aggiornamento delle metriche stabilite, sfruttando le funzionalità offerte dai fogli di calcolo.

4.4) Processo di miglioramento

4.4.1) Introduzione

Il **processo di miglioramento** ha lo scopo di stabilire, valutare, misurare, controllare e migliorare in modo continuo i processi del ciclo di vita del software adottati dal progetto.

Attraverso un approccio sistematico e iterativo, il processo consente di incrementare l'efficacia e l'efficienza delle attività svolte, garantendo l'allineamento agli standard di qualità e alle esigenze del progetto.

4.4.2) Attività del processo

Il processo di miglioramento è articolato nelle seguenti attività principali:

1. **Inizializzazione del processo** - Questa attività si occupa di definire e documentare i processi organizzativi adottati dal gruppo. Il presente documento assolve a questo compito, fungendo da riferimento ufficiale per la definizione dei processi.
2. **Valutazione del processo** - Questa attività consiste nello sviluppo, nella documentazione, nell'utilizzo e nella storicizzazione⁷ delle procedure di valutazione dei processi. Al fine di garantirne efficacia ed efficienza, tali procedure sono sottoposte a revisioni periodiche basate sui dati raccolti durante l'esecuzione del progetto.
3. **Miglioramento del processo** - In seguito alle attività di valutazione, vengono pianificate, attuate e documentate le azioni di miglioramento dei processi. A tal fine vengono raccolti e analizzati i seguenti dati: 1. Storico del processo; 2. Dati tecnici; 3. Risultati delle valutazioni; 4. Performance del processo.

Tali attività sono eseguite ciclicamente durante l'intero svolgimento del progetto e costituiscono la base del miglioramento continuo dei processi organizzativi.

4.4.3) Procedure operative

4.4.3.1) Ciclo PDCA

Le attività del processo di miglioramento si inseriscono all'interno del ciclo di miglioramento continuo **PDCA (Plan-Do-Check-Act)**:

- **Plan**: individuazione delle criticità nei processi attuali tramite retrospettive e definizione di nuovi standard o modifiche alle norme per risolverle;
- **Do**: applicazione sperimentale delle nuove procedure definite durante il normale svolgimento delle attività operative;
- **Check**: misurazione delle prestazioni tramite le metriche e confronto dei dati raccolti con lo storico precedente per valutare se il cambiamento ha portato benefici;
- **Act**: aggiornamento formale delle **Norme di Progetto** e consolidamento delle procedure che si sono rivelate efficaci, scartando quelle inefficienti.



Figura 1: Rappresentazione del ciclo PDCA applicato ai processi organizzativi

⁷Tramite registrazione.

4.4.3.1.1) Guida alla Retrospettiva

Il nostro gruppo ha ritenuto fondamentale l'uso della **retrospettiva**, soprattutto nell'ambito della pianificazione e del monitoraggio delle attività descritte nel [Piano di Progetto](#).

1. A turno, ogni membro del gruppo condivide le attività svolte e segnala eventuali problemi riscontrati.
2. Si discutono le criticità o i dubbi emersi durante le attività.
3. Alla fine della riunione, con l'assegnazione dei ruoli, le task vengono ridistribuite considerando eventuali difficoltà o problematiche segnalate.

Tutti i membri partecipano attivamente alla definizione delle azioni di miglioramento, sia sulle attività di documentazione sia sulla gestione delle issue e delle infrastrutture.⁸

⁸Il gruppo adotta una filosofia di miglioramento continuo, accettando che all'inizio alcune procedure possano non essere perfette o contenere errori, ma mantenendo la massima trasparenza per favorire l'apprendimento e l'ottimizzazione dei processi.

4.4.3.1.2) Automiglioramento sulla Requirements and Technology Baseline

Problema Generale	Soluzione Generale
La mancanza di processi strutturati per la gestione delle attività, la rendicontazione e la nomenclatura degli artefatti genera inconsistenze, stime poco affidabili e un elevato rischio di errori manuali	Adozione di strumenti e convenzioni condivise — issue tracking su GitHub, template standardizzati e fogli di rendicontazione asincrona — per centralizzare il controllo, uniformare il lavoro e ridurre il carico operativo nelle sessioni di retrospettiva
L'aggiornamento manuale di metriche, grafici e codici identificativi introduce inefficienze ricorrenti e aumenta la probabilità di dati non allineati o numerazioni inconsistenti	Sviluppo di script di automazione che rielaborano e aggiornano metriche, grafici e mappature dei codici in modo sistematico, riducendo l'effort manuale e garantendo la coerenza dei dati nel tempo
La granularità insufficiente nella pianificazione e l'integrazione tardiva sul branch principale rallentano l'avanzamento e rendono difficile valutare lo stato reale del progetto	Introduzione di branch feature con integrazione controllata e revisione continua del backlog a ogni sprint, suddividendo le attività complesse in sotto-issue collegate per mantenere stime affidabili e una base di codice stabile
La rigidità nell'assegnazione dei ruoli e la gestione informale delle modifiche riducono la flessibilità operativa e aumentano il rischio di inconsistenze nella documentazione e nel versionamento	Ogni membro può svolgere attività su un secondo ruolo dichiarato esplicitamente, mentre tutti sono tenuti a rispettare le Norme di Progetto per modifiche, tracciamento e versionamento, garantendo qualità e coerenza nel tempo
La scarsa familiarità con le tecnologie proposte e la gestione manuale di documentazione e navigazione rallentano il lavoro e aumentano il rischio di errori e inefficienze ricorrenti	Combinazione di apprendimento collaborativo per le tecnologie nuove e sviluppo di script di automazione, affiancati da un miglioramento strutturale del sito per rendere i documenti più facilmente reperibili

Tabella 6: Automiglioramento: Requirements and Technology Baseline

4.4.3.1.3) Automiglioramento sulla Technology Baseline

Problema Generale	Soluzione Generale
L'assenza di una strategia strutturata per la gestione del codice sorgente aumenta il rischio di conflitti, regressioni e integrazioni problematiche sul branch principale, rendendo difficile tracciare l'avanzamento delle singole funzionalità	Adozione di un flusso di lavoro basato su branch dedicati per ogni feature, con integrazione sul branch principale esclusivamente tramite pull request soggette a revisione da parte di almeno un altro membro del team
La scrittura dei test posticipata a fasi avanzate dello sviluppo genera accumulo di debito tecnico e riduce la capacità di individuare tempestivamente regressioni o comportamenti inattesi nel codice	Introduzione della pratica di scrittura dei test fin dalle prime fasi di sviluppo, garantendo la verificabilità continua del codice prodotto e riducendo il costo della correzione degli errori nel tempo
La mancanza di riferimenti architetturali condivisi porta a scelte implementative eterogenee tra i membri del team, compromettendo la coerenza strutturale e la manutenibilità del prodotto nel lungo periodo	Adozione della Specifica Tecnica come riferimento vincolante per la scrittura del codice, assicurando che le scelte implementative siano allineate alle linee guida architetturali e alle convenzioni definite dal gruppo

Tabella 7: Automiglioramento: Technology Baseline

4.5) Processo di formazione

4.5.1) Introduzione

Il **processo di formazione** ha lo scopo di garantire che tutti i membri del team acquisiscano le competenze necessarie per contribuire efficacemente alle attività del progetto. Si tratta di un percorso continuo che combina sessioni strutturate e apprendimento autonomo, permettendo ai partecipanti di affrontare le sfide reali del progetto con consapevolezza e autonomia.

4.5.2) Scopo del processo

Il processo mira a:

- Facilitare l'acquisizione di competenze tecniche e organizzative;
- Favorire la comprensione dei documenti, degli strumenti e delle procedure del progetto;
- Promuovere l'apprendimento collaborativo e la condivisione delle conoscenze tra i membri del team.

4.5.3) Attività del processo

Le attività principali del processo di formazione comprendono:

1. **Sessioni formative guidate su strumenti e metodologie** del progetto;
2. **Studio dei documenti di progetto e dei materiali** forniti dal docente;

3. **Esercitazioni pratiche** per l'applicazione delle conoscenze acquisite;
4. **Apprendimento autonomo e auto-valutazione** delle competenze.

4.5.4) Procedure operative

4.5.4.1) Apprendimento libero

Non sempre un programma di apprendimento rigoroso riesce a rispondere in modo efficace alle esigenze reali e immediate che un membro del team può incontrare nello svolgimento di un task. Perciò, il processo prevede la possibilità di **formazione autonoma**, lasciando libertà ai membri del gruppo di approfondire tematiche o strumenti specifici secondo le proprie necessità.

4.5.4.2) Apprendimento in gruppo

Poiché alcuni processi possono essere molto complessi e richiedere tempi lunghi, il gruppo può decidere di **riunirsi in sottogruppi** o dividere il task tra i membri⁹.

Questi sottogruppi possono essere impiegati anche per:

- Lo sviluppo del **Proof of Concept**;
- Lo studio di nuove tecnologie;
- La realizzazione di prototipi per capire quali tecnologie funzionano meglio.

Lo **studio in gruppo** ha il vantaggio di:

- Favorire il confronto tra i membri;
- Migliorare la comprensione dei contenuti complessi;
- Velocizzare i tempi di apprendimento e di completamento dei task.

4.5.4.3) Trasferimento dei ruoli

Al momento del **cambio dei ruoli**, il membro che cede il ruolo ha la responsabilità di trasmettere conoscenze e competenze acquisite al nuovo titolare del ruolo.

In particolare:

- Se il membro uscente ritiene di aver sviluppato tecniche o conoscenze importanti, deve **formare inizialmente il nuovo membro**;
- Passare il **testimone e le conoscenze**, rendendo più semplice l'ingresso del nuovo responsabile nel ruolo;
- Garantire continuità operativa, riducendo errori dovuti alla mancata familiarità con le procedure e gli strumenti del progetto.

Questo approccio consente di **mantenere l'efficienza del team** e di valorizzare l'esperienza accumulata durante il progetto.

4.5.4.4) Rendicontazione delle ore

Essendo un progetto svolto per la prima volta, ogni attività formativa viene registrata anche in termini di ore non produttive. Tali ore comprendono:

⁹Ad esempio, questa opzione è stata utilizzata per studiare il materiale fornito dall'azienda.

- **Studio dei documenti e dei materiali forniti** dal docente;
- **Preparazione dei documenti di progetto** nel modo più accurato possibile;
- **Apprendimento** delle procedure interne e gestione degli strumenti.

Questa organizzazione permette di monitorare l'impegno formativo dei membri del team e di garantire che ciascuno possa acquisire competenze adeguate per svolgere le proprie attività in modo efficace.

5) Metriche

5.1) Introduzione

Le metriche rappresentano strumenti fondamentali per misurare, controllare e migliorare sia il processo di sviluppo sia i prodotti realizzati.

Il tracciamento delle metriche viene effettuato all'interno del documento: [Piano di Qualifica](#).

Nelle Norme di Progetto sono considerate due principali categorie di metriche:

1. **Metriche di qualità di processo** – Consentono di valutare l'efficacia, l'efficienza e la correttezza dei processi di sviluppo, monitorando la pianificazione, l'esecuzione e la gestione delle attività.
2. **Metriche di qualità di prodotto** – Misurano le caratteristiche intrinseche dei deliverable.

In particolare

Funzionalità:

Valuta la capacità del software di fornire correttamente le funzionalità richieste dai requisiti, assicurando completezza e coerenza rispetto alle specifiche definite. In particolare sono considerate:

- **Adeguatezza:**

Presenza di funzioni appropriate per i compiti che deve svolgere,

- **Accuratezza:**

Capacità di fornire risultati nel modo stabilito nei requisiti,

- **Interoperabilità:**

La capacità del prodotto di interagire con altri sistemi definiti,

- **Conformità:**

Il prodotto aderisce a determinati standard di dominio,

- **Sicurezza:**

Non vi sono falle di sicurezza ,

Affidabilità:

Misura la capacità del software di operare senza guasti in condizioni previste, garantendo comportamenti consistenti e riducendo al minimo malfunzionamenti.

- **Maturità:**

Capacità dell'applicazione di evitare blocchi a seguito di errori nel software.

- **Tolleranza agli errori:**

Capacità di mantenere determinati livelli di prestazione in caso di errori.

- **Recuperabilità:**

Capacità del software di ripristinare il proprio livello di prestazioni e di recuperare i dati direttamente interessati a seguito di un guasto o di un errore.

- **Aderenza :**

Capacità di aderire a standard di affidabilità.

Efficienza:

Indica l'ottimizzazione delle risorse e la rapidità di risposta del software alle richieste, valutando tempi di esecuzione, throughput e utilizzo delle risorse disponibili.

- **Comportamento temporale:**

Capacità di soddisfare le richieste con un tempo di risposta adeguato.

- **Utilizzo delle risorse:**

Capacità di usare le risorse in modo proporzionale alle richieste.

- **Conformità:**

Aderenza del prodotto a standard riguardanti l'efficienza.

Usabilità:

Rileva quanto il software sia intuitivo e facile da utilizzare, considerando la semplicità delle interazioni, la facilità di apprendimento e la correttezza delle operazioni da parte degli utenti.

- **Comprensibilità:**

Facilità di comprensione del prodotto.

- **Apprendibilità:**

Facilità di apprendimento delle funzionalità.

- **Operabilità:**

Semplicità di utilizzo del prodotto.

- **Attrattività:**

Capacità di fornire un'esperienza utente gradevole.

- **Conformità:**

Aderenza a standard di usabilità.

Manutenibilità:

Misura quanto facilmente il software può essere modificato o esteso senza introdurre errori, tenendo conto della complessità del codice, della modularità e della facilità di intervento sugli artefatti.

- **Analizzabilità:**

Facilità di ispezione del codice con lo scopo di cercare errori

- **Modificabilità:**

Facilità nell'apportare modifiche.

- **Stabilità:**

Capacità del prodotto di limitare effetti indesiderati derivanti da modifiche o aggiunte al prodotto.

- **Testabilità:**

Semplicità di testing del prodotto.

Portabilità:

Rappresenta la facilità con cui il prodotto può essere spostato da un ambiente di lavoro ad un altro.

- **Adattabilità:**

Facilità di adattamento del prodotto ad ambienti diversi.

- **Installabilità:**

Facilità di installazione del prodotto su un ambiente di sviluppo.

- **Conformità:**

Aderenza alle convenzioni sulla portabilità.

- **Sostituibilità:**

Facilità di migrazione da un prodotto precedente al prodotto oggetto del progetto.

L'adozione sistematica delle metriche permette di identificare aree di miglioramento, garantire trasparenza e mantenere elevati standard di qualità durante l'intero ciclo di vita del progetto.

5.1.1) Nomenclatura delle Metriche

La nomenclatura adottata è la seguente:

TIPO_METRICA-##

dove:

- TIPO_METRICA =
 - **MPC** per le metriche di qualità del processo
 - **MPD** per le metriche di qualità del prodotto
- **##** è un contatore progressivo

6) Metriche di Qualità del Processo

6.1) Processi primari

I processi primari riguardano le attività di sviluppo del software, come requisiti, progettazione, implementazione, integrazione e manutenzione. Per valutarne andamento ed efficacia si utilizzano metriche di processo che permettono di monitorare tempi, costi e progressi, evidenziando eventuali scostamenti rispetto agli obiettivi.

6.1.1) Fornitura

Codice	Metrica	Valore accettabile	Valore ottimo
MPC01	Planned Value	≥ 0	$\leq BAC$
MPC02	Earned Value	$\geq PV * 0.75$	$\geq PV$
MPC03	Actual Cost	$0 \leq AC \leq 1.2 * EV$	$\leq EV$
MPC04	Schedule Performance Index	≥ 0.9	≥ 1.0
MPC05	Cost Performance Index	≥ 0.9	≥ 1.0
MPC06	Estimate at Completion	$\leq 1.1 * BAC$	$\leq BAC$
MPC07	To Complete Performance Index	~ 1.0	≤ 1.0
MPC08	Estimate to Complete	$\leq (BAC - AC) * 1.1$	$\leq BAC - AC$

Tabella 8: Metriche processo di Fornitura

Planned Value

- **Codice:** MPC01

- **Descrizione:**

Rappresenta il valore del lavoro che avrebbe dovuto essere completato entro una certa data, secondo il piano di progetto approvato.

- **Formula:**

$$PV = BAC * \% \text{ di lavoro pianificata}$$

BAC Sta per budget at completion

- **Interpretazione:**

- Valore accettabile: ≥ 0
- Valore ottimo: $\leq BAC$

- **Calcolo:**

Si può rappresentare con il totale di ore produttive previste

Earned Value

- **Codice:** MPC02
- **Descrizione:**
Rappresenta il valore del lavoro completato rispetto al budget previsto.
Molto utile per monitorare l'andamento effettivo delle attività di progetto.
- **Formula:**

$$EV = BAC * \% \text{ di lavoro completata}$$

BAC Sta per budget at completion

- **Interpretazione:**
 - Valore accettabile: $\geq PV * 0.75$
 - Valore ottimo: $\geq PV$
- **Calcolo:**
Si può rappresentare con il totale di ore produttive effettive

Actual Cost

- **Codice:** MPC03
- **Descrizione:**
Rappresenta il costo effettivo sostenuto.
Risulta utile nel verificare che il lavoro svolto sia in linea con le aspettative.
- **Formula:**

$$AC = \text{Costo sostenuto nello sprint}$$

- **Interpretazione:**
 - Valore accettabile: $0 \leq AC \leq 1.2 * EV$
 - Valore ottimo: $\leq EV$

Schedule Performance Index

- **Codice:** MPC04
- **Descrizione:**
Rappresenta il rapporto tra il valore del lavoro completato e il lavoro pianificato.
- **Formula:**
$$\text{Schedule Performance Index} = \frac{\text{Earned Value}}{\text{Planned Value}}$$
- **Interpretazione:**
 - Valore accettabile: ≥ 0.9
 - Valore ottimo: ≥ 1.0

Cost Performance Index

- **Codice:** MPC05
- **Descrizione:**
Rappresenta il rapporto tra il valore del lavoro completato e il costo effettivamente sostenuto.
- **Formula:**
$$\text{Cost Performance Index} = \frac{\text{Earned Value}}{\text{Actual Cost}}$$
- **Interpretazione:**
 - Valore accettabile: ≥ 0.9
 - Valore ottimo: ≥ 1.0

Estimate at Completion

- **Codice:** MPC06
- **Descrizione:**
Rappresenta il costo da sostenere per il completamento delle attività sulla base della performance attuali.
- **Formula:**
$$\text{Estimate at Completion} = \frac{\text{Budget at completion}}{\text{Cost Performance Index}}$$
- **Interpretazione:**
 - Valore accettabile: $\leq 1.1 * \text{BAC}$
 - Valore ottimo: $\leq \text{BAC}$

To Complete Performance Index

- **Codice:** MPC07
- **Descrizione:**
Rappresenta l'efficienza richiesta in futuro per completare il lavoro entro il termine previsto.
- **Formula:**
$$\text{Estimate at Completion} = \frac{\text{Budget at Completion} - \text{Earned Value}}{\text{Budget at Completion} - \text{Actual Cost}}$$
- **Interpretazione:**
 - Valore accettabile: ~ 1.0
 - Valore ottimo: ≤ 1.0

Estimate To Complete

- **Codice:** MPC08
- **Descrizione:**
Rappresenta il costo ancora da sostenere per il completamento delle attività
- **Formula:**
$$\text{Estimate To Complete} = \text{Estimate at Completion} - \text{Actual Cost}$$
- **Interpretazione:**
 - Valore accettabile: $\leq (\text{BAC} - \text{AC}) * 1.1$
 - Valore ottimo: $\leq \text{BAC} - \text{AC}$

6.1.2) Sviluppo

Codice	Metrica	Valore accettabile	Valore ottimo
MPC09	Requirements Stability Index	≥ 0.7	1.0

Tabella 9: Metriche processo di Sviluppo

Requirements Stability index

- **Codice:** MPC09
- **Descrizione:**
Misura i cambiamenti apportati ai requisiti nel tempo.
- **Formula:**
$$\text{Requirements Stability Index} = \frac{\text{NRI} - (\text{NC} + \text{NRC} + \text{NRA})}{\text{NRI}}$$
- Legenda:
 - Numero Requisiti Iniziali = NRI.
 - Numero di Cambiamenti = NC.
 - Numero di Requisiti Cancellati = NRC.
 - Numero di Requisiti Aggiunti = NRA.
- **Interpretazione:**
 - Valore accettabile: ≥ 0.7
 - Valore ottimo: 1.0

6.2) Processi di supporto

Questi includono attività che garantiscono controllo, tracciabilità e affidabilità del processo stesso, come la verifica, la validazione, la gestione della configurazione, la documentazione tecnica e l'assicurazione qualità. Questi processi consentono di monitorare e ridurre gli scostamenti rispetto agli standard pianificati. Le metriche associate ai processi di supporto sono definite per consentire una valutazione oggettiva della conformità e del controllo delle attività di supporto rispetto ai processi e alle procedure stabilite.

6.2.1) Documentazione

Codice	Metrica	Valore accettabile	Valore ottimo
MPC10	Indice di Gulpease	≥ 60	≥ 70
MPC11	Correttezza ortografica	≤ 1	$= 0$

Tabella 10: Metriche processo di Documentazione

Indice di Gulpease

- **Codice:** MPC10

- **Descrizione:**

L'Indice di Gulpease misura la leggibilità di un testo.

Valuta il grado di istruzione necessario alla comprensione del contenuto.

- **Formula:**

$$IG = 89 + \frac{300 * (\text{numero di frasi}) - 10 * (\text{numero di lettere})}{\text{numero di parole}}$$

- **Interpretazione:**

- Valore accettabile: ≥ 60
- Valore ottimo: ≥ 70

- **Calcolo:**

Per il calcolo il gruppo ha sviluppato una propria automazione sfruttando le funzionalità di Typst. Questo permette una forte integrazione e analisi di dettaglio più fine.

Correttezza Ortografica

- **Codice:** MPC11
- **Descrizione:**
Misura la qualità della documentazione
- **Formula:**

$$\text{Correttezza Ortografica} = 1000 * \frac{\text{numero di errori ortografici}}{\text{Numero di parole}}$$

- **Interpretazione:**
 - Valore accettabile: ≤ 1
 - Valore ottimo: $= 0$

6.2.2) Verifica

Codice	Metrica	Valore accettabile	Valore ottimo
MPC12	Test Success Rate	$\geq 90\%$	100%

Tabella 11: Metriche processo di Verifica

Test Success Rate

- **Codice:** MPC12
- **Descrizione:**
Rappresenta la percentuale di test automatizzati che vengono superati.
- **Formula:**

$$\text{Test success rate} = \frac{\text{Numero di Test superati}}{\text{Numero totale dei test}}$$

- **Interpretazione:**
 - Valore accettabile: $\geq 90\%$
 - Valore ottimo: 100%

6.3) Processi organizzativi

Riguardano il miglioramento continuo del processo, la definizione degli standard interni, la gestione della qualità complessiva e lo sviluppo delle competenze del perso-

nale. Questi processi assicurano la sostenibilità e la maturità del modello di sviluppo nel tempo. Le metriche associate ai processi organizzativi sono definite per consentire una valutazione oggettiva della conformità e dell'efficacia dei processi di gestione e governance interna.

6.3.1) Gestione dei processi

Codice	Metrica	Valore accettabile	Valore ottimo
MPC13	Time Efficiency	$\geq 80\%$	$\geq 100\%$
MPC14	Process Lead Time	$\geq 90\%$	$= 100\%$

Tabella 12: Metriche processo di Gestione dei Processi

Time Efficiency

- **Codice:** MPC13

- **Descrizione:**

Rappresenta il rapporto tra le ore previste e le ore effettivamente impiegate, misurato in modo cumulativo sprint per sprint. Un valore inferiore a 1 indica che il team ha impiegato più ore del pianificato.

- **Formula:**

$$\text{Time Efficiency} = \frac{\text{Ore previste cumulative}}{\text{Ore effettive cumulative}}$$

- **Interpretazione:**

- Valore accettabile: $\geq 80\%$
- Valore ottimo: $\geq 100\%$

Process Lead Time

- **Codice:** MPC14

- **Descrizione:**

Stima la durata finale del progetto in settimane, basandosi sulla velocità reale del team misurata tramite lo Schedule Performance Index.

- **Formula:**

$$\text{TimeEAC} = \frac{\text{Settimane Pianificate}}{\text{SPI}}$$

- **Interpretazione:**

- Valore accettabile: $\geq 90\%$
- Valore ottimo: $= 100\%$

7) Metriche di Qualità del Prodotto

7.1) Funzionalità

Codice	Metrica	Valore accettabile	Valore ottimo
MPD01	Requisiti obbligatori soddisfatti	100%	100%
MPD02	Requisiti desiderabili soddisfatti	$\geq 50\%$	$\geq 75\%$
MPD03	Requisiti opzionali soddisfatti	$\geq 0\%$	$\geq 50\%$

Tabella 13: Metriche di funzionalità del prodotto

Requisiti obbligatori soddisfatti

- **Codice:** MPD01
- **Descrizione:**
Rappresenta la percentuale di requisiti obbligatori soddisfatti.
È utile a monitorare il grado di soddisfacimento dei requisiti essenziali.
- **Formula:**

$$RObbS = \frac{\text{Numero di requisiti obbligatori soddisfatti}}{\text{Numero di requisiti obbligatori}}$$

- **Interpretazione:**
 - Valore accettabile:100%
 - Valore ottimo:100%

Requisiti Desiderabili soddisfatti

- **Codice:** MPD02
- **Descrizione:**
Rappresenta la percentuale di requisiti desiderabili soddisfatti.
È utile a monitorare il grado di soddisfacimento dei requisiti desiderabili.
- **Formula:**

$$RDesS = \frac{\text{Numero di requisiti desiderabili soddisfatti}}{\text{Numero di requisiti desiderabili}}$$

- **Interpretazione:**
 - Valore accettabile: $\geq 50\%$
 - Valore ottimo: $\geq 75\%$

Requisiti opzionali soddisfatti

- **Codice:** MPD03
- **Descrizione:**
Rappresenta la percentuale di requisiti opzionali soddisfatti.
È utile a monitorare il grado di soddisfacimento dei requisiti opzionali.
- **Formula:**

$$ROpzS = \frac{\text{Numero di requisiti opzionali soddisfatti}}{\text{Numero di requisiti opzionali}}$$

- **Interpretazione:**
 - Valore accettabile: $\geq 0\%$
 - Valore ottimo: $\geq 50\%$

7.2) Affidabilità

Codice	Metrica	Valore accettabile	Valore ottimo
MPD04	Failure Density	≤ 0.5	≤ 0.2
MPD05	Statement Coverage	$\geq 80\%$	$\geq 95\%$
MPD06	Branch Coverage	$\geq 70\%$	$\geq 90\%$

Tabella 14: Metriche di affidabilità del prodotto

Failure Density

- **Codice:** MPD04
- **Descrizione:**
Numero di failure per 1000 linee di codice (KLOC)
- **Formula:**

$$\text{Failure Density} = \frac{\text{Numero di Failure}}{\text{KLOC}}$$

- **Interpretazione:**
 - Valore accettabile: ≤ 0.5
 - Valore ottimo: ≤ 0.2

Statement Coverage

- **Codice:** MPD05
- **Descrizione:**
Percentuale di istruzioni del codice coperte da test automatizzati.
- **Formula:**

$$\text{Statement Coverage} = \frac{\text{Istruzioni Testate}}{\text{Istruzioni Totali}}$$

- **Interpretazione:**
 - Valore accettabile: $\geq 80\%$
 - Valore ottimo: $\geq 95\%$

Branch Coverage

- **Codice:** MPD06
- **Descrizione:**
Percentuale di rami di codice coperte da test automatizzati.
- **Formula:**

$$\text{Branch coverage} = \frac{\text{Numero di Branch coperti da test}}{\text{Numero di Branch totali}}$$

- **Interpretazione:**
 - Valore accettabile: $\geq 70\%$
 - Valore ottimo: $\geq 90\%$

7.3) Usabilità

Codice	Metrica	Valore accettabile	Valore ottimo
MPD07	User Error Rate	$\leq 3\%$	$\leq 1\%$
MPD08	Time to Complete Task	10 minuti	5 minuti

Tabella 15: Metriche di usabilità del prodotto

User Error Rate

- **Codice:** MPD07
- **Descrizione:**
Misura quanto spesso un utente fa errori durante l'uso del prodotto.
- **Formula:**

$$\text{User error rate} = \frac{\text{Errori Totali}}{\text{Azioni Totali}}$$

- **Interpretazione:**
 - Valore accettabile: $\leq 3\%$
 - Valore ottimo: $\leq 1\%$

Time To Complete Task

- **Codice:** MPD08
- **Descrizione:**
Tempo medio per completare un'attività.
- **Formula:**

$$\frac{\text{Tempo Ottimista} + 4 * \text{Tempo Probabile} + \text{Tempo Pessimista}}{6}$$

- **Interpretazione:**
 - Valore accettabile: 10 minuti
 - Valore ottimo: 5 minuti

7.4) Efficienza

Codice	Metrica	Valore accettabile	Valore ottimo
MPD09	Response Time	$\leq 3 \text{ sec}$	$\leq 1 \text{ sec}$
MPD10	CPU Utilization	$\leq 35\%$	$\leq 20\%$
MPD11	Memory Utilization	$\leq 4,0 \text{ GB}$	$\leq 1,5 \text{ GB}$

Tabella 16: Metriche di efficienza del prodotto¹⁰

¹⁰I valori soglia di questa sezione sono espressi in termini assoluti, non come percentuali relative.

Response Time

- **Codice:** MPD09
- **Descrizione:**
Misura il tempo medio impiegato dal prodotto per rispondere a una richiesta.
- **Interpretazione:**
 - Valore accettabile: ≤ 3 sec
 - Valore ottimo: ≤ 1 sec

CPU Utilization

- **Codice:** MPD10
- **Descrizione:**
Misura l'utilizzo di CPU lato client.
- **Interpretazione:**
 - Valore accettabile: $\leq 35\%$
 - Valore ottimo: $\leq 20\%$
- **Calcolo:**
Le percentuali fanno riferimento ai seguenti processori:
 - Intel Core Ultra 5
 - AMD Ryzen 5 5500

Memory Utilization

- **Codice:** MPD11
- **Descrizione:**
Misura l'utilizzo di memoria RAM lato client.
- **Interpretazione:**
 - Valore accettabile: $\leq 4,0$ GB
 - Valore ottimo: $\leq 1,5$ GB

7.5) Manutenibilità

Codice	Metrica	Valore accettabile	Valore ottimo
MPD12	Cyclomatic Complexity	≤ 10	≤ 8
MPD13	Instability Index	$I \geq 0.7$ $\vee$ $I \leq 0,30$	$I \geq 0.85$ $\vee$ $I \leq 0,15$
MPD14	Coefficient of Coupling	≤ 0.4	≤ 0.2
MPD15	Code Smell	≤ 10	≤ 5
MPD16	Code Coverage	$\geq 80\%$	$\geq 90\%$

Tabella 17: Metriche manutenibilità del prodotto

Cyclomatic Complexity

- **Codice:** MPD12
- **Descrizione:**
Misura la complessità del codice in base al numero di percorsi linearmente dipendenti.
- **Formula:**

$$\text{Cyclomatic complexity} = E - N + 2P$$

***E*:**

Numero di archi nel grafo di controllo

***N*:**

Numero di nodi nel grafo di controllo

***P*:**

Numero di componenti connesse

- **Interpretazione:**
 - Valore accettabile: ≤ 10
 - Valore ottimo: ≤ 8

Instability Index

- **Codice:** MPD13
- **Descrizione:**
Misura la resilienza di un modulo al cambiamento.
- **Formula:**

$$\text{Instability index} = \frac{C_e}{C_e + C_a}$$

C_e:

Numero di classi esterne da cui il modulo dipende

C_a:

Numero di classi esterne che dipendono dal modulo

- **Interpretazione:**
 - Valore accettabile: $I \geq 0.7$
✓
 $I \leq 0,30$
 - Valore ottimo: $I \geq 0.85$
✓
 $I \leq 0,15$

Coefficient of coupling

- **Codice:** MPD14
- **Descrizione:**
Misura il coupling tra le componenti del sistema.
- **Formula:**

$$\text{Coefficient of coupling} = \frac{\text{Numero di dipendenze}}{\text{Numero di componenti}}$$

- **Interpretazione:**
 - Valore accettabile: ≤ 0.4
 - Valore ottimo: ≤ 0.2

Code Smell

- **Codice:** MPD15
- **Descrizione:**
Misura il rapporto tra codice smell e KLOC.
- **Formula:**
$$\text{Code smell} = \frac{\text{Numero di codice smell}}{\text{KLOC}}$$
- **Interpretazione:**
 - Valore accettabile: ≤ 10
 - Valore ottimo: ≤ 5

Code Coverage

- **Codice:** MPD16
- **Descrizione:**
Rappresenta la percentuale di codice coperto da test automatizzati.
- **Formula:**
$$\text{Code Coverage} = \frac{\text{Linee di codice testate}}{\text{Linee di codice totali}}$$
- **Interpretazione:**
 - Valore accettabile: $\geq 80\%$
 - Valore ottimo: $\geq 90\%$